

A
PROJECT REPORT ON
Sign language Interpreter
SUBMITTED BY
Kunal Vishwakarma
UNDER GUIDENCE OF
PROFESSOR: SHRADDHA KOKATE
EXAMINATION BSC.IT,
SEMESTER V-VI

University of Mumbai



UNIVERSITY OF MUMBAI
DEPARTMENT OF INFORMATION TECHNOLOGY
GHANSHYAMDAS SARAF COLLEGE OF ARTS AND COMMERCE
(Affiliated to university of Mumbai)
MUMBAI-400064
MAHARASHTRA
2024 – 2025

SIGN LANGUAGE INTERPRETER

A Project Report

Submitted in partial fulfillment of the
Requirements for the award of the Degree of

BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

By

KUNAL VISHWAKARMA

Seat Number: 3029866

Under the esteemed guidance of
PROF. SHRADDHA KOKATE



Ghanshyamdas Saraf
college of arts & commerce
EDUCATION EMPOWERS

DEPARTMENT OF INFORMATION TECHNOLOGY
GHANSHYAMDAS SARAF COLLEGE OF ARTS AND COMMERCE

(Affiliated to university of Mumbai)

MUMBAI-400064

MAHARASHTRA

2024 – 2025

PROFORMA FOR THE APPROVAL PROJECT PROPOSAL

(Note: All entries of the proforma of approval should be filled up with appropriate and complete information. Incomplete proforma of approval in any respect will be summarily rejected.)

PNR No.:

Roll no:

1. Name of the Student

2. Title of the Project

3. Name of the Guide

4. Teaching experience of the Guide

5. Is this your first submission? Yes ☐ No ☐

Signature of the Student

Signature of the Guide

Date:

Date:

Signature of the Coordinator

Date:

GHANSHYAMDAS SARAF COLLEGE OF ARTS AND COMMERCE

(Affiliated to university of Mumbai)

MUMBAI-MAHARASHTRA-400064

DEPARTMENT OF INFORMATION TECHNOLOGY



Ghanshyamdas Saraf
college of arts & commerce
EDUCATION EMPOWERS

CERTIFICATE

This is to certify that the project entitled, "**Sign Language Interpreter**", is bonafied work of **KUNAL VISHWAKARMA** bearing Seat.No: _____ submitted in partial fulfillment of the requirements for the award of degree of BACHELOR OF SCIENCE in INFORMATION TECHNOLOGY from University of Mumbai.

Internal Guide

Coordinator

External Examiner

Date:

College Seal

Abstract

Sign Language Interpreter is portion of the population is unable to understand it. This project addresses the communication gap between D&M individuals and the wider community by providing a user-friendly interface that

Keywords:

- **Communication Barrier** – Many people cannot understand sign language, creating a communication gap.
- **Target Audience** – Designed for Deaf and Mute (D&M) individuals and the wider community.
- **Project Goal** – To bridge the gap by providing a sign language interpreter that facilitates communication.
- **User-Friendly Interface** – Ensures accessibility and ease of use for all users.
- **Real-Time Interpretation** – Converts sign language into text or speech for better understanding.
- **Inclusivity & Accessibility** – Aims to make interactions smoother for people with hearing and speech impairments.
- **Technology Integration** – Likely involves AI, machine learning, and computer vision to recognize and translate signs.

ACKNOWLEDGMENT

I would like to express our special thanks and gratitude to our project guide Prof. Shraddha Kokate Ma'am for guiding me to do the project work on time and giving us all support and guidance, which made complete our project duly. We are extremely thankful to her for providing such nice support and guidance.

I am also thankful for and fortunate enough to get constant encouragement, support and guidance from the teachers of information Technology who helped me in successfully completing our project work.

DECLARATION

I hereby declare that the project entitled, “**Sign Language Interpreter**” done at **Ghanshyamdas Saraf College of Arts and Commerce**, has not been in any case duplicated to submit to anyother universities for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university.

The project is done in partial fulfillment of the requirements for the award of degree of **BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)** to be submitted as final semester project as part of my curriculum.

Kunal Vishwakarma

TABLE OF CONTENTS

Chapter 1: Introduction	1
1.1 Background.....	1
1.2 Objectives	1
1.3 Purpose and Scope	1
1.3.1 Purpose	2
1.3.2 Scope	2
1.4 Applicability	2
1.5 Organization of Report.....	3
Chapter 2: Survey of Technologies	4
2.1 Software.....	4,5,6
2.1.1 For Designing	7
2.1.2 For Development	7
2.1.3 For Project Management.....	8
2.2 Programming Languages	9
2.3 Frontend & Backend	10
2.3.1 Frontend Technologies.....	10
2.3.2 Backend Technologies	11
2.4 Algorithms	12,13
Chapter 3: Requirement and Analysis.....	14
3.1 Problem Definition.....	14
3.2 Requirements Specifications	14
3.2.1 Functional Requirements	14
3.2.2 Non-Functional Requirements	15
3.3 Planning and Scheduling.....	15
3.3.1 Gantt Chart	16
3.3.2 PERT Chart	16
3.4 .1 Software Requirements	16
3.4.2 Hardware Requirements.....	17

3.5 Conceptual Models.....	17
3.5.1 Use Case Diagram	17
3.5.2 Activity Diagram	17-23
3.5.3 State Transition Diagram.....	24
3.5.4 Sequence Diagram	24
3.5.5 Sequence Diagram	25
3.5.6 Class Diagram	27
3.5.7 Event Table.....	28
Chapter 4: System Design.....	29
4.1 Basic Modules.....	29
4.2 Data Designs	31
4.2.1 Schema Design	31
4.3 Process Design.....	32
4.3.1 Flow Chart.....	31
4.4 Algorithm Design.....	33
4.5 UI Designs	35
Chapter 5: Implementation and Testing	36
5.1 Implementation Approaches	37
5.2 Coding Details and Code Efficiency	38
5.2.1 Coding.....	38-48
5.2.2 Code Efficiency	48
5.3 Testing Approaches.....	49-50
5.3.1 Unit Testing	50-53
5.3.2 Integration testing	53-55
5.3.3 System Testing.....	55,56
Chapter 6: Results and Discussion.....	57
6.1 Test Reports	57
6.1.1 Informal Testing	57
6.1.2 Formal Testing.....	58
6.2 User Documentation.....	59-66

Chapter 7: Conclusions.....	67
7.1 Conclusion	67
7.2 Limitations of the System.....	67
7.3 Future Scope of the Project	68
REFERENCES	68-69

List of Tables

Sr.no	Table Name	Page no.
1	Comparing between Figma and Canva	5
2	Comparing between EdrawMax and Lucidchart	6
3	Comparing between VSCode and PyCharm	7
4	Comparing between PyQt and Tkinter	11
5	Comparing between Flask and Django	12
6	Algorithms	13
7	Event Table	36
8	SST Table	51
9	TTS Table	51
10	Lesson UITable	51
11	Beginner Table	51
12	Intermediate table	52
13	Learning Table	53
14	Event table	53
15	Integration Testing	53
16	Summary of unit Testing	58
17	Summary of Integration Testing	58

List of Figures

Figure no.	Figure Name	Page no.
1	Gantt Chart	16
2	Pert Chart	16
3	Use Case Diagram	18
4	Activity Diagram	19
5	State Transition Diagram	24
6	Sequence Diagram	25
7	Class Diagram	26

CHAPTER NO. 1: INTRODUCTION

1.1. Background

Sign language is an ancient and natural way to communicate, especially for deaf and mute (D&M) individuals who can't use spoken languages. This project proposes using a neural network, specifically a convolutional neural network (CNN), to recognize American Sign Language (ASL) finger spelling through hand gestures captured by a camera. The CNN will be trained to identify and predict these gestures, helping D&M people express their thoughts visually

Around 430 million deaf people worldwide use sign languages to communicate, but not everyone knows how to understand these gestures. This project aims to create a user- friendly interface that can interpret ASL, making it easier for D&M individuals to interact and be included in their communities. The goal is to improve their communication and access to services, ensuring they enjoy equal rights.

1.2. Objective

1. **Real-time translation:** The ability to translate written language instantly, as it's being communicated.
2. **Support Multiple languages:** The system can handle and translate Multiple language.
3. **Context understanding:** The system can comprehend the context or meaning behind words and sentences, not just translating them word-for-word.
4. **Gesture understanding:** By The system can recognize and interpret gestures made by people.
5. **Translate sign language:** The system can understand and translate sign language into written language, and vice versa.

1.3. Purpose and scope

1.3.1. Purpose

- This system will be highly beneficial for both deaf and mute individuals, as well as for

people who do not understand sign language. By simply using hand gestures, a person can communicate effectively through this system. The software will capture the image of the hand gesture, identify what it signifies using a trained model, and then convert that gesture into text. This means that deaf and mute individuals can express their thoughts and needs without relying on others to interpret their sign language.

- Additionally, the system will be helpful for people who do not know sign language. When they see the text output, they will understand what the deaf or mute person is trying to convey. Providing an text message that can understood by everyone. This output in text makes communication seamless and inclusive, bridging the gap between those who use sign language and those who do not.

1.3.2. Scope

- The system detects hand gestures of only one person at a time.
- It focuses solely on detecting users' hand gestures.
- The camera must be positioned in front of the user to capture gestures effectively.
- Internet connectivity is required for real-time data transmission, analysis, essential for providing immediate feedback.

1.4. Applicability

The project aims to improve communication for deaf and mute individuals by translating their hand gestures into understandable language, bridging the gap between them and the hearing community. By facilitating better understanding of ASL, the project promotes inclusivity, allowing deaf and mute people to participate more fully in social, educational, and professional environments. The system can be used as a learning aid for both deaf individuals to practice ASL and for hearing individuals to learn and understand ASL, fostering better mutual communication. It can help D&M individuals access services more easily, such as healthcare, banking, and customer support, where communication barriers might otherwise exist.

1.5. Organization of Report

The subsequent chapters of the project report will focus on the survey of different technologies which include a comparison of different platforms, languages, software, hardware etc.

The Survey is followed by the analysis of requirements resulting in the generation of requirement specifications and a schedule of activities. It also includes the conceptual design that visualizes the features and operations that can be performed on the system.

The final chapter of the project report includes the system design that describes desired features and operations in detail, including screen layouts, business rules, process diagrams, pseudo code, and other documentation.

CHAPTER NO. 2: SURVEY OF TECHNOLOGY

2.1. Software

2.1.1. Designing

- **Figma**
 - The Figma is a cloud-based design tool primarily used for UI/UX design.
 - Multiple users can work on the same design file simultaneously. Changes made by one user are instantly visible to others.
 - Users can create reusable components (buttons, icons, etc.) and maintain design systems with consistent styles and assets.
 - Offers features like auto-layout, grids, and smart selection to manage and organize design elements efficiently.
 - Simplifies the layout process and ensures that designs are well-organized and aligned, reducing manual adjustments and enhancing design consistency.

- **Canva**
 - Canva is a user-friendly design tool that is ideal for creating marketing materials, social media graphics, presentations, and more.
 - Canva provides an intuitive drag-and-drop interface that simplifies the design process.
 - Canva offers a wide range of pre-designed templates for various formats such as social media posts, presentations, flyers, and more.
 - Users can customize templates and design elements including text, images, colors, and shapes.
 - Provides flexibility to tailor designs according to specific needs and branding requirements, making it easy to create unique and personalized content.

Feature	Figma	Canva
Target Audience	Professional designers and development teams.	General users, marketers, and non-designers.
Real-Time Collaboration	Yes, multiple users can edit simultaneously.	Limited collaboration features.
Design Focus	High-fidelity UI/UX design and prototyping.	Graphic design for various media formats.
Templates	Minimal, emphasis on custom design.	Extensive library of pre-made templates.
Ease of Use	Advanced features with a steeper learning curve.	Intuitive and user-friendly.

- **Why Figma over Canva for Sign Language Interpreter?**

While Canva is an excellent tool for creating marketing materials and quick and quick designs, Figma stands out as the best choice for designing the Sign Language Interpreter platform due to its advanced design capabilities, real-time collaboration, prototyping features, and robust design system management. Figma's features align perfectly with the needs of a comprehensive and interactive e-learning platform, making it the ideal tool for this project.

- **EdrawMax**

- EdrawMax is a versatile and comprehensive diagramming software.
- EdrawMax provides an extensive library of over 50,000 vector symbols and a wide array of templates. This variety supports the creation of various diagrams, such as flowcharts, mind maps, and organizational charts.
- Users can quickly start their projects with pre-designed templates and symbols, streamlining the diagram creation process.
- The software features an intuitive drag-and-drop interface, making it accessible to users with varying levels of experience.
- Simplifies the diagramming process and reduces the learning curve for new users.

- **Lucidchart**

- Lucidchart is a web-based diagramming tool known for its emphasis on

real-time collaboration and integration with various productivity tools.

- Enables multiple users to work on diagrams simultaneously, with changes visible in real-time.
- Facilitates teamwork and ensures that all collaborators are on the same page, improving project efficiency.
- Offers a wide selection of templates to help users get started quickly.
- Provides a fast and easy way to create diagrams by starting with pre-designed layouts.

Feature	EdrawMax	Lucidchart
Target Audience	Targets professionals across various industries needing versatile diagramming tools.	Focused on business and tech teams, popular with project managers and engineers.
Real-Time Collaboration	Supports collaboration, but real-time features aren't as advanced.	Excels in real-time collaboration with seamless simultaneous editing
Design Focus	Emphasizes versatility and detailed, professional-grade diagrams.	Focuses on simplicity, ideal for business processes and clear visual communication.
Templates	Offers a wide range of templates across various industries	Provides robust templates, especially for business and technical diagrams.
Ease of Use	Powerful but has a steeper learning curve for beginners.	User-friendly and easy to pick up, with a simple drag-and-drop interface.

- **Why EdrawMax over Lucidchart for Sign Language Interpreter?**

EdrawMax is the optimal choice for Sign Language Interpreter due to its extensive features, customization options, and offline accessibility. The vast symbol library and support for multiple import and export formats provide the flexibility needed for creating detailed and diverse diagrams. The ability to use the software across different platforms and offline ensures that team members can work on their designs without interruption. These advantages make EdrawMax the best designing software for the Sign Language Interpreter project, catering to all the specific requirements of creating an interactive and comprehensive e-learning platform.

2.1.2. Development

- **Visual Studio Code (VS Code)**
 - Visual Studio Code, developed by Microsoft is a lightweight and highly customizable code editor. It supports a wide range of programming languages and offers an extensive library of extensions to enhance its functionality.
 - VSCode is a lightweight yet powerful code editor that provides essential features for coding, debugging, and version control.
 - Offers a comprehensive development environment while maintaining speed and performance.
 - VSCode provides syntax highlighting for various programming languages and IntelliSense, which includes code completion, parameter info, and quick info.
- **PyCharm**
 - PyCharm, developed by JetBrains, is an IDE specifically designed for Python development. It offers a comprehensive set of tools and features tailored for Python and web development, including support for Django.
 - PyCharm offers advanced code editing features including syntax highlighting, code completion, and error detection.
 - Enhances coding efficiency and accuracy by providing intelligent suggestions and identifying potential issues as you type.

Aspects	Visual Studio Code (VS Code)	PyCharm
Features and Functionality	Highly extensible with numerous extensions, built-in terminal, Git integration, customizable.	Comprehensive set of built-in features for Python, advanced debugging, testing, database tools.
Pricing	Free and open-source.	Community edition (free), Professional edition (paid).
Ease of Use	Easier to start, simple extension installation.	Steeper learning curve, extensive built-in tools.
Integration and Ecosystem	Integrates well with various languages and tools, versatile editor for multi-language projects.	Specialized for Python, deep integration with Python-specific tools and frameworks.

- **Why VS Code over PyCharm for Sign Language Interpreter?**

While PyCharm offers a comprehensive set of tools specifically for Python development, VS Code stands out as the best choice for the development of the Sign Language Interpreter platform due to its lightweight performance, extensive extension marketplace, integrated terminal, real-time collaboration features, cross-platform consistency, and high customizability. These features align perfectly with the needs of a modern, efficient, and collaborative development environment, making VS Code the ideal tool for this project.

2.1.3. Project Management

- **Notion**

- Notion is an all-in-one workspace that combines notes, tasks, databases, and calendars in a single platform. It is highly customizable and can be tailored to fit various project management needs, from simple task tracking to complex project planning.
- Notion combines notes, tasks, databases, and collaboration tools into a single platform.
- Centralizes various aspects of work and personal organization, reducing the need for multiple separate tools and improving efficiency.
- Notion offers a wide range of pre-designed templates for different use cases, such as project management, meeting notes, and personal planning.
- Speeds up setup and ensures consistency by providing ready-made structures that can be customized to fit specific needs.

- **Trello**

- Trello is a visual project management tool that uses boards, lists, and cards to help teams organize tasks and projects. It is known for its simplicity and ease of use, making it a popular choice for managing tasks and workflows.
- Trello uses Kanban-style boards to visually organize tasks and projects. Boards consist of lists and cards that represent different stages of a workflow.
- Provides a clear and intuitive way to manage and track tasks, improving organization and visibility of project progress.

- Within each board, tasks are organized into lists (e.g., To Do, In Progress, Done) and represented by cards.
- Allows for easy categorization and movement of tasks through different stages, enhancing workflow management and task tracking.

- **Why Notion over Trello for Sign Language Interpreter?**

While Trello is a powerful and user-friendly project management tool, Notion stands out as the best choice for managing the Sign Language Interpreter project due to its comprehensive all-in-one solution, advanced customization options, rich text and documentation capabilities, powerful database management, real-time collaboration features, and offline access. These features align perfectly with the needs of a complex and dynamic project like Sign Language Interpreter, making Notion the ideal project management tool for this endeavor.

2.2. Programming Language

- **Python:**

Python is a high-level, interpreted programming language known for its simplicity, readability, and versatility. Created by Guido van Rossum and first released in 1991, Python has become one of the most popular programming languages in the world due to its clear syntax and broad applicability.

- Python has a clear and readable syntax that is easy for beginners to learn and understand.
- Facilitates a smooth learning curve for new programmers and enhances productivity with simple and expressive code.
- Python abstracts complex details of the computer's hardware and provides high-level constructs for programming.
- Allows developers to focus on solving problems rather than managing low-level details, making development faster and more efficient.

- **Java:**

Java is a high-level, object-oriented programming that was developed by Sun Microsystems, with its first version released in 1995. Java is designed to be portable, secure, and robust, making it a widely-used language for building a variety of applications, from mobile and web applications to large-scale enterprise systems.

- Java is designed around the principles of object-oriented programming, including encapsulation, inheritance, and polymorphism.
 - Promotes code reuse, modularity, and organization, making it easier to manage and maintain complex codebases.
 - Java code is compiled into bytecode, which can be executed on any device or operating system with a Java Virtual Machine (JVM).
 - Allows Java applications to run on any platform without modification, achieving the "Write Once, Run Anywhere" philosophy.
- **Why Python over Java for Sign Language Interpreter?**

Python's simplicity, rich ecosystem, and rapid development capabilities make it a strong candidate for Sign Language Interpreter, particularly given the platform's need for advanced features like AI-driven recommendations and interactive tools. While Java is a powerful language with its own strengths, Python's advantages in terms of development speed, community support, and integration with modern technologies make it a more suitable choice for this project.

2.3. Fronted and Backend

2.3.1. Fronted

- **PyQT:** PyQt and PySide are both Python bindings for the Qt application framework, allowing developers to create cross-platform graphical user interfaces (GUIs). **PyQt**, developed by Riverbank Computing, offers a rich set of features and widgets for building complex applications, but requires a GPL or commercial license, which can involve costs for proprietary software. **PySide**, developed by The Qt Company, provides similar functionality but under the more permissive LGPL license, making it more suitable for closed-source projects. Both libraries enable the development of professional-grade, modern applications with advanced features and a native look and feel across Windows, macOS, and Linux, though they differ slightly in their licensing and some API details.
- **Tkinter:** Tkinter is the standard GUI toolkit included with Python, providing a simple and lightweight framework for developing desktop applications. It offers a basic set of widgets and tools for creating straightforward graphical interfaces, making it ideal for beginners and small projects. Tkinter is bundled with Python, requiring no additional installation, and is known for its ease of use and quick setup. However, it lacks the advanced features and modern look found in more comprehensive libraries like PyQt and PySide, making it less suitable for complex or highly customized applications.

Comparison:

Features	PyQT	Tkinter
Complexity and Features	Comprehensive set of widgets and advanced functionality for complex applications.	Basic set of widgets for simpler applications.
Modern Look and Feel	Modern and highly customizable appearance, supports theming and styling.	Outdated default appearance, limited customization
Learning Curve	Steeper learning curve due to extensive feature set and complex API.	Easier to learn and use, suitable for beginners and simple projects.
Cross-platform Support	Consistent appearance and functionality across Windows, macOS, and Linux.	Cross-platform support with slight variations in widget appearance across OSes.

2.3.2. Backend

- **Flask:** Flask is a lightweight and micro web framework for Python, designed to be simple and easy to use. It is built with a minimalistic approach, offering the core features needed to build web applications and APIs without imposing any dependencies or project structure. Flask allows developers to choose the components they want to integrate, making it highly flexible and modular. With its simplicity and straightforward design, Flask is ideal for small to medium-sized applications, rapid prototyping, and projects that require customization. It comes with built-in support for unit testing, RESTful request dispatching, and Jinja2 templating. Flask's extensibility through extensions enables the addition of various functionalities as needed, allowing developers to scale their applications while maintaining control over the components used.
- **Django:** Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design. It follows the “batteries-included” philosophy, providing a wide range of built-in features out of the box, such as an ORM (Object-Relational Mapping) for database interaction, authentication, an admin interface, and a powerful template engine. Django emphasizes reusability, pluggability of components, and the principle of "don't repeat yourself" (DRY). This framework is suitable for large and complex web applications, as it helps developers adhere to best practices and avoid common security pitfalls. Django's comprehensive documentation and active community support further enhance its appeal for projects that require a robust and scalable solution.

Comparison:

Features	Flask	Django
Type	Microframework.	Full-stack framework.
Flexibility	Highly flexible and modular	Less flexible but very feature- rich.
Built-in Features	Minimal, relies on extensions.	Extensive built-in features.
Project Structure	Simple, developer-defined.	Predefined, with a conventional structure.

2.4 . Algorithm Used

2.4.1. Convolutional Neural Network

- **CNN:** Convolutional Neural Network is a deep learning technique which is developed from the inspiration of visual cortex which are the fundamental blocks of human vision. It is observed from the research that, the human brain performs a large-scale convolutions to process the visual signals received by eyes, based on this observation CNNs are constructed and observed to be outperforming all the prominent.
- classification techniques .Two major operations performed in CNN are convolution ($wT * X$) and pooling($\max()$) and these blocks are wired in a highly complex fashion to mimic the human brain .The neural network is constructed in layers, where the increase in the number of layers increases the network complexity and is observed to improve the system accuracy. The CNN architecture consists of three operational blocks which are connected as a complex architecture.
- The functional blocks of Convolutional Neural Network:
 1. Convolutional Layer
 2. Max Pooling Layer
 3. Fully-Connected Layer

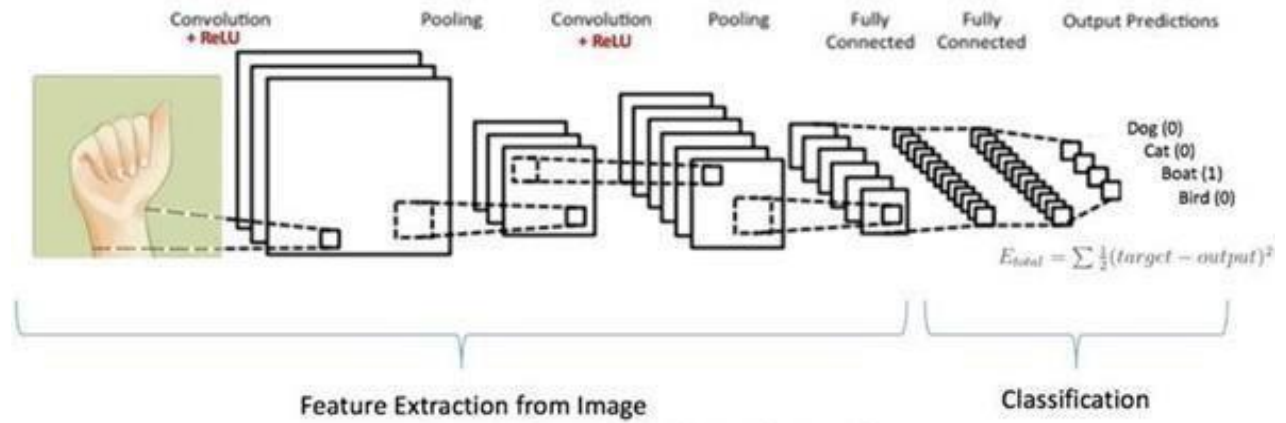


Figure 8: Convolutional Neural Network

CHAPTER NO. 3: REQUIREMENTS AND ANALYSIS

3.1. Problem Definition

In the current scenario people those are deaf and hard-of-hearing face challenges of translating between sign language (used by the deaf and hard-of-hearing) and spoken or written language (used by hearing individuals). The purpose is to create a desktop **Sign language interpretation** application that can bridge the communication gap between sign language users and non-sign language users by providing accurate and real-time translation between sign language and spoken/written language. This is essential for ensuring accessibility, inclusivity, and effective communication. This application help users by providing services such as translate text into sign language and vice versa, allowing deaf/hearing people individuals to communicate effectively in real-time, whether in conversations, meetings, or events.

3.2. Requirement Specification

3.2.1. Functional Requirements

- **Webcam Access**
 - The application should be able to access the user's webcam to capture real-time video input for sign language gestures.
- **Real-Time Gesture Analysis**
 - The system should analyze the live video feed to detect and interpret sign language gestures in real-time.
- **Gesture-to-Text Translation**
 - The application should translate detected gestures into text output, allowing non-sign language users to understand.
- **Output Format Selection**
 - Users should be able to select a preferred output format (text) for translated gestures.
- **Performance Optimization**
 - The software should run efficiently in the background, without significantly impacting system performance or slowing down other applications.

- **User-Friendly Interface**
 - The App should be easy to use and user-friendly.

3.2.1 Non-Functional Requirements

- **Performance:**
 - The sign language interpretation should operate in real-time with minimal delay, providing immediate feedback to the user. The application should be optimized to use CPU and memory efficiently, ensuring smooth performance alongside other desktop applications.
- **Scalability:**
 - The The software should be scalable to work with various camera angles, hand sizes and lighting conditions It should also support the expansion to recognize more sign languages and gestures in the future.
- **Accuracy:**
 - The system should be highly accurate in recognizing signs and gestures, minimizing false positives (incorrectly identifying a sign) and false negatives (failing to detect a valid sign). It should differentiate between similar signs with precision, ensuring reliable interpretation.
- **Compatibility:**
 - The application should be compatible with operating systems (Windows) and work seamlessly with webcam models or external cameras.

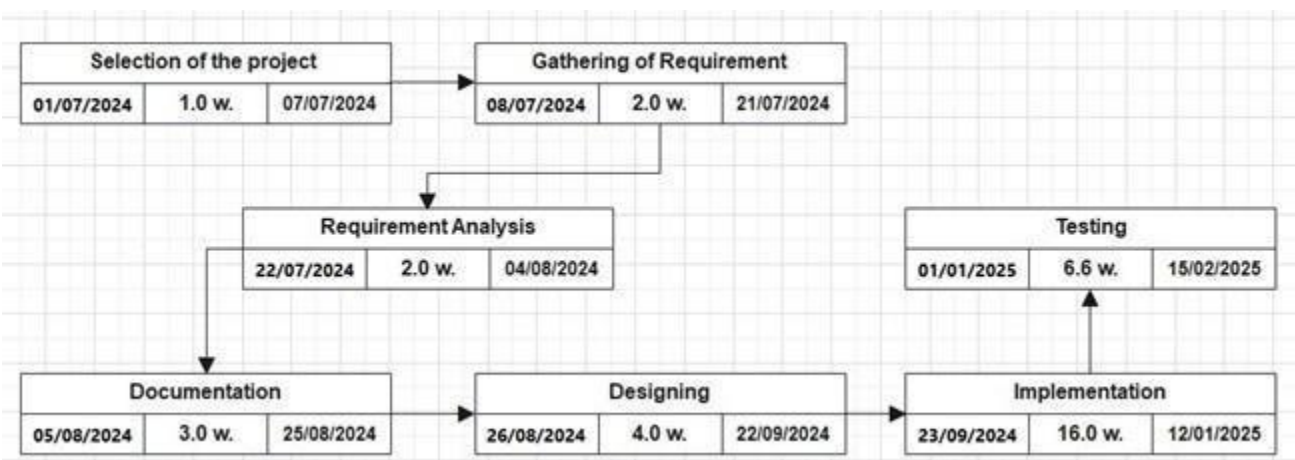
3.3. Planning and Scheduling

- Number of Personnel: 2
- Estimated Start Date: 1st july,2024
- Estimated End Date: 15 February,2025

Gantt CHART



PERT CHART



3.4. Software and Hardware Requirements

3.4.1 Software requirements

- **Operating System:**
Windows 10.
- **Programming Language:**
Python (core programming language).
- **Development Tools:**

Visual Studio Code (IDE for writing and debugging Python code).

- **Libraries and Frameworks:**

PyQt or PySide (for building graphical user interfaces, GUI). TensorFlow or PyTorch (for machine learning models and sign language recognition). OpenCV (for image processing and handling real-time video feed). Keras (for building and training neural networks).

- **Additional Tools:**

Pip (for managing Python packages). Python virtual environment (for handling dependencies effectively).

3.4.2 Hardware requirements

- **Computer:**

Processor: Intel i5 or higher. RAM: 8GB minimum (16GB recommended for smoother multitasking). Storage: 256GB SSD (recommended for faster performance).

- **Camera:**

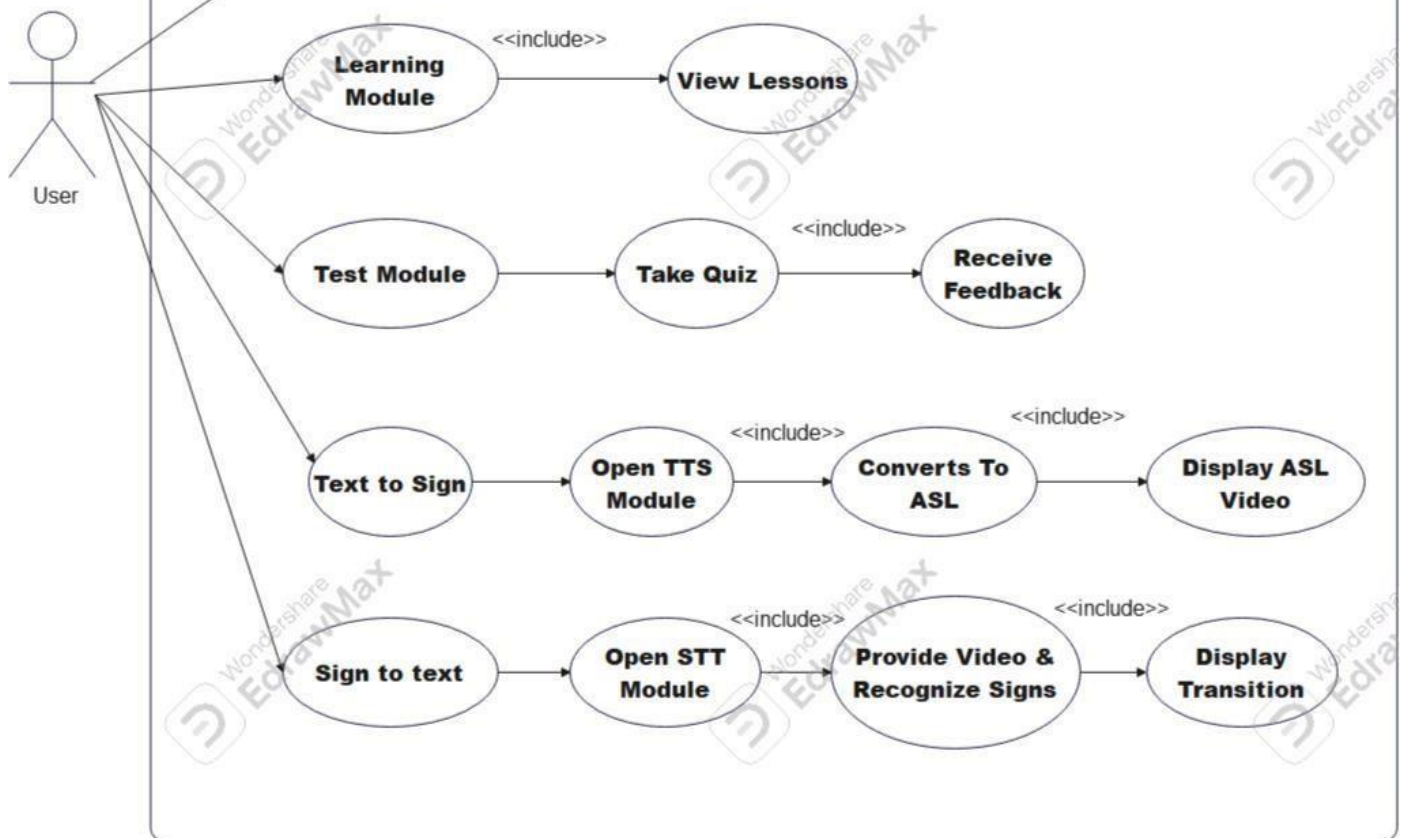
A high-definition (HD) Camera for capturing hand gestures and signs in real-time. 1080p resolution is preferable for accurate gesture detection.

- **Internet Connection:**

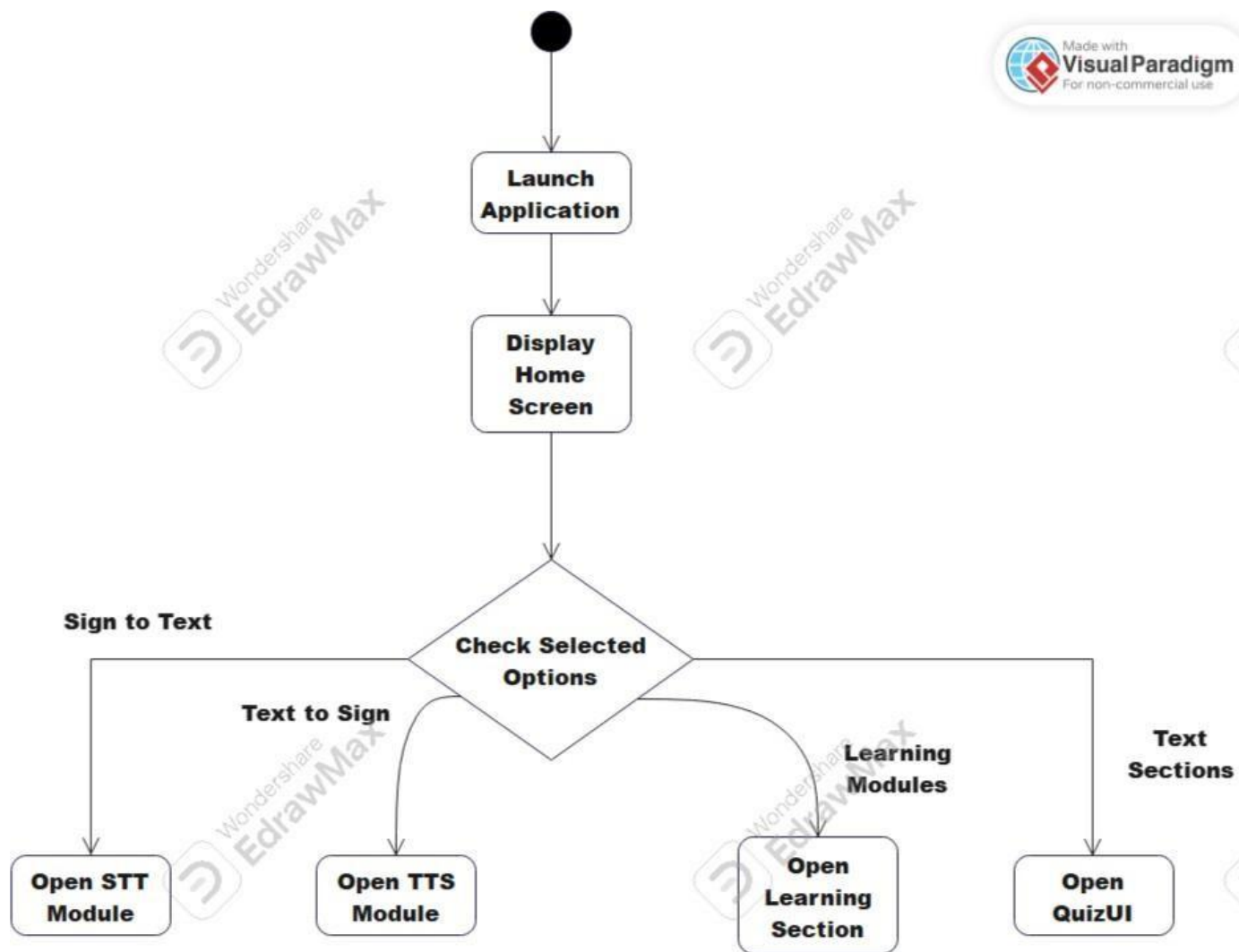
If needed for remote libraries or updates

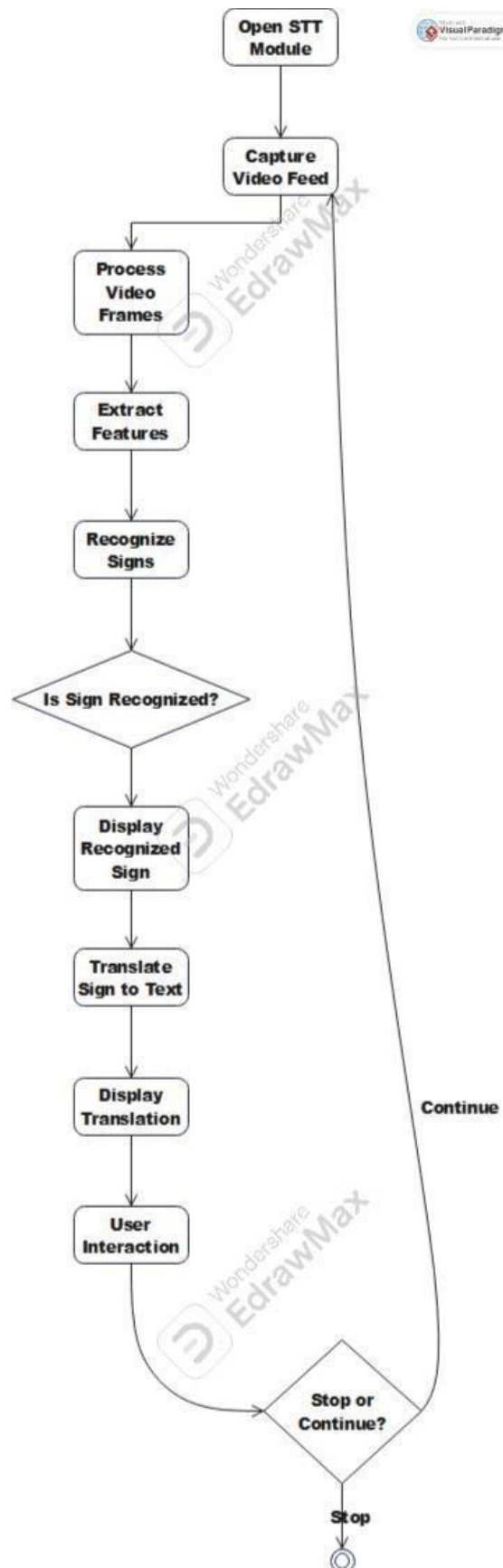
3.5. Conceptual Tools

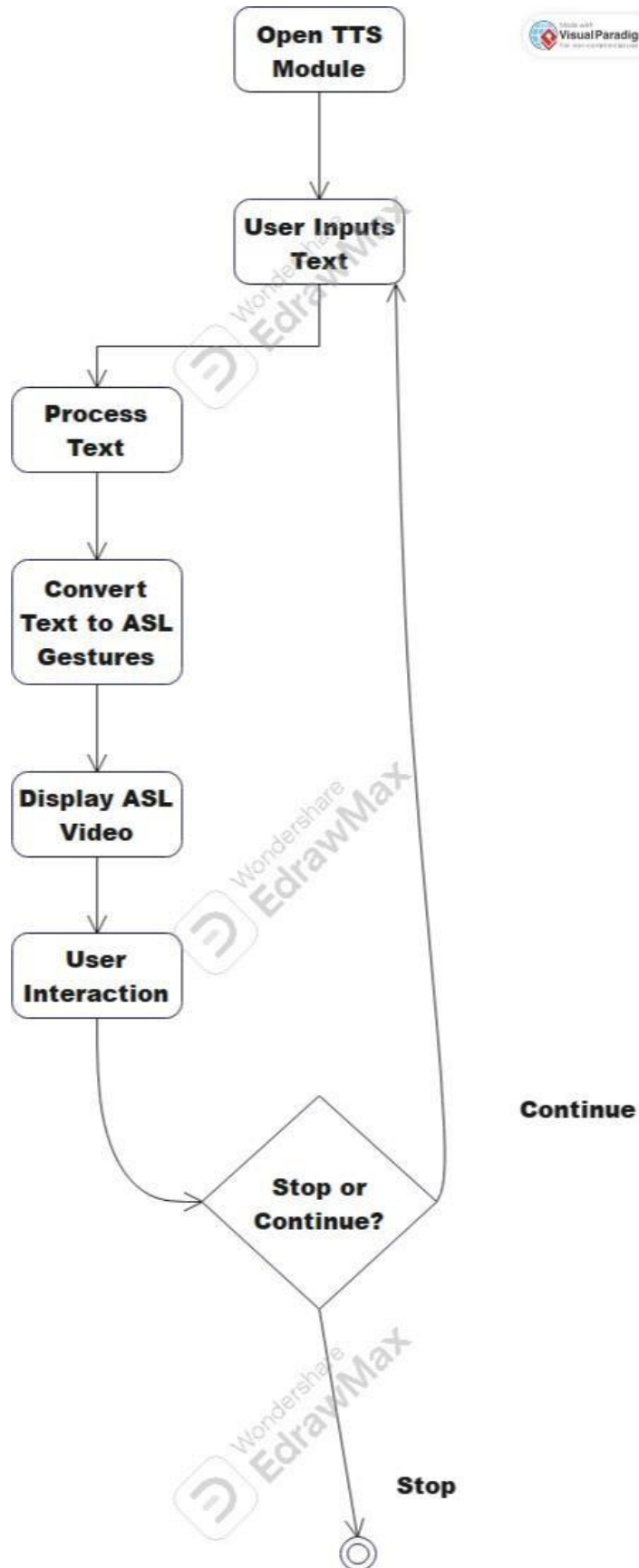
3.5.1. Use Case Diagram.

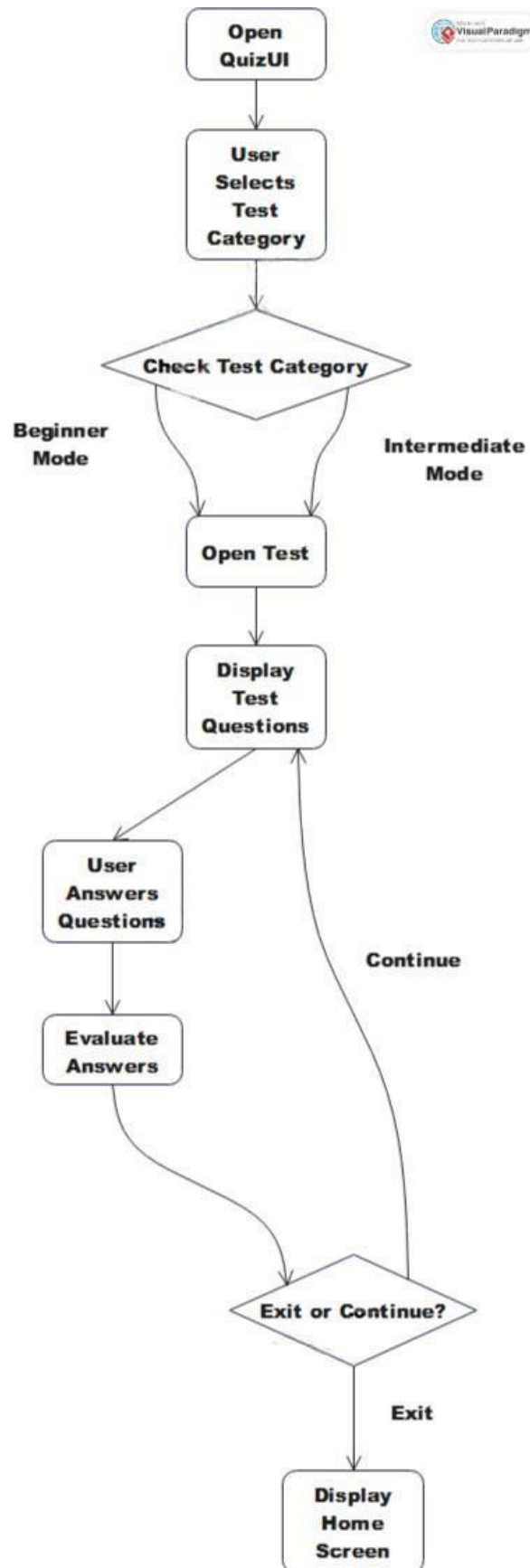


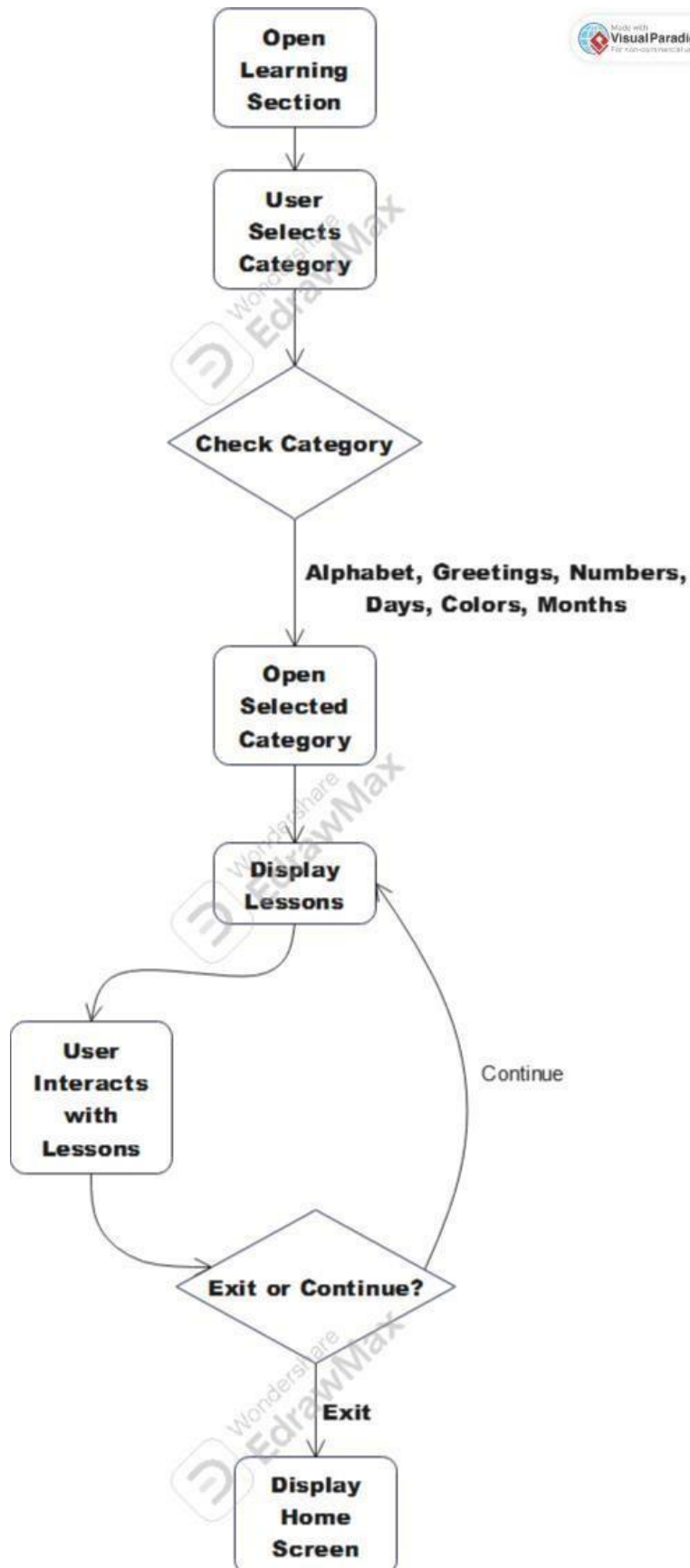
3.5.2 Activity Diagram.



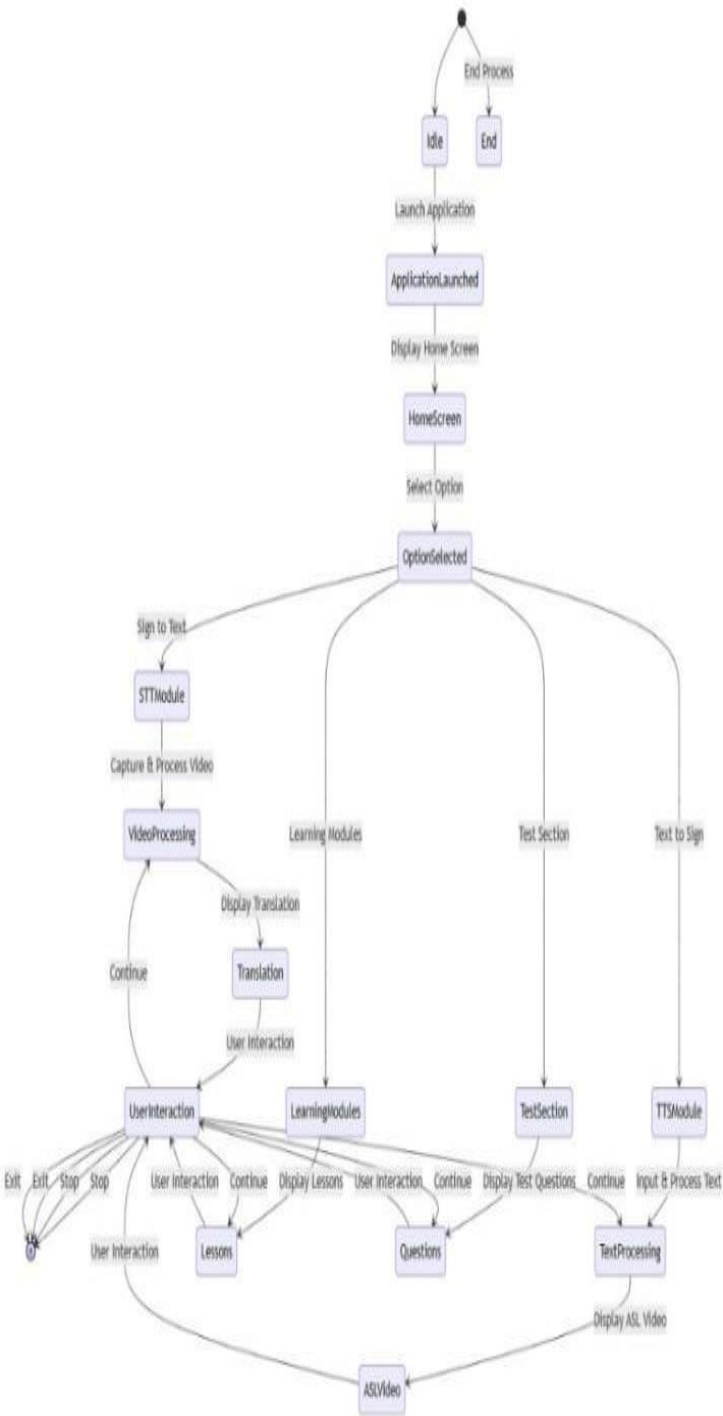




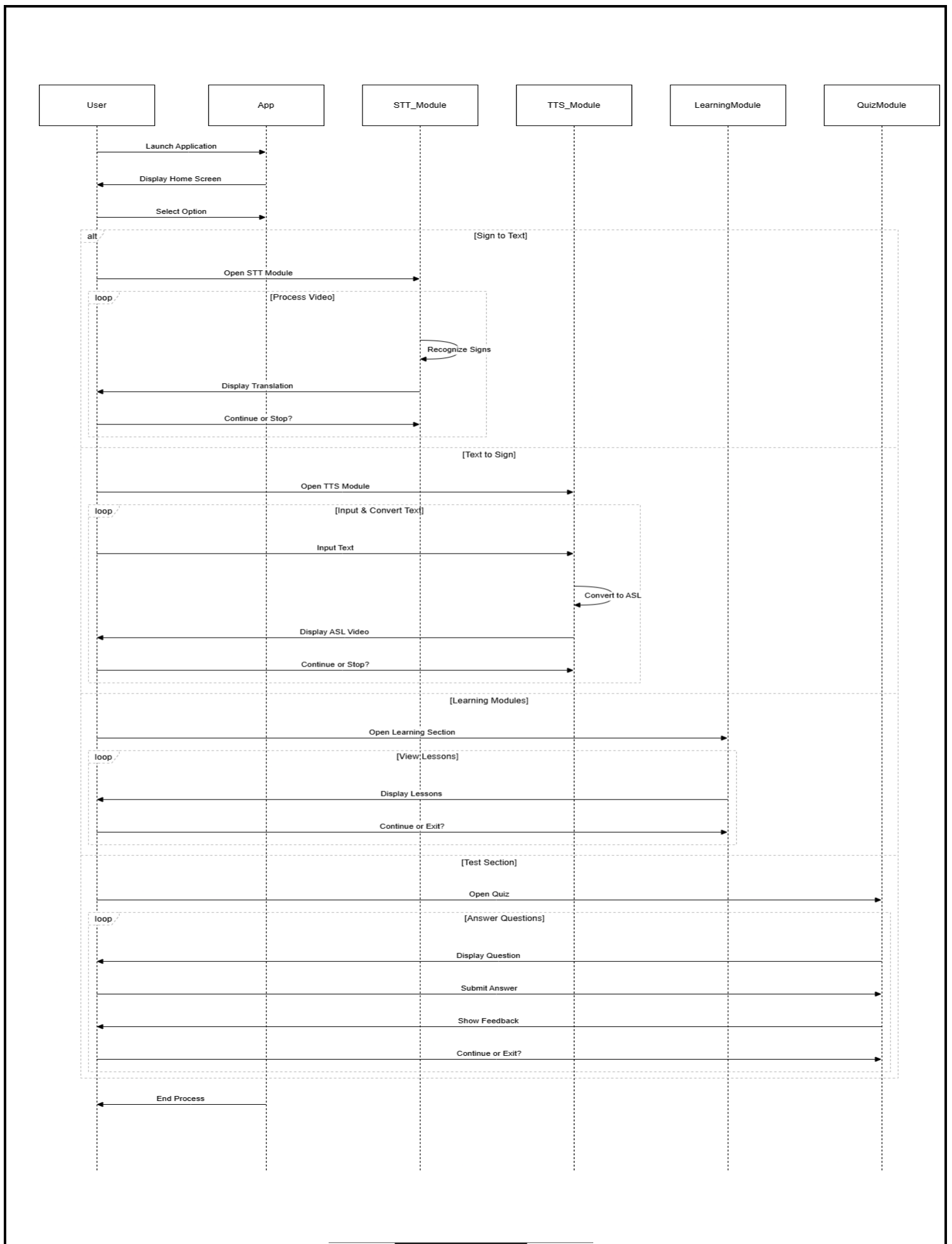




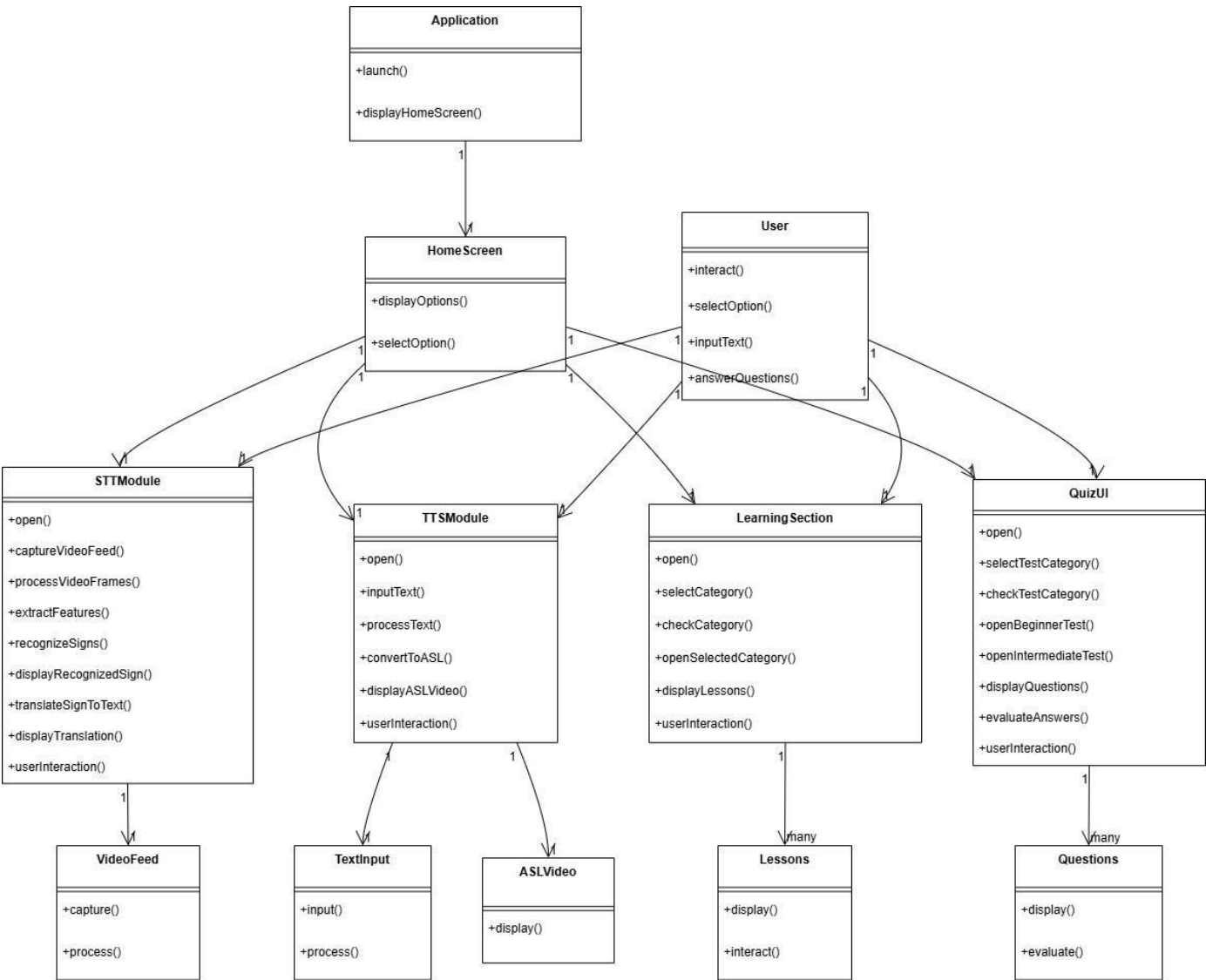
3.5.3 State Transition Diagram



3.5.4. Sequence Diagram



3.5.5 Class Diagram



3.5.6 Event table

Event Name	Trigger	Source	Activity	Response	Destination
Launch Application	User opens the application	User	Initialize application	Display home screen	Home Screen
Select Option	User selects an option	User	Process selected option	Open corresponding module	STT/TTS/Learning/Test
Start Video Capture	User initiates video capture	User Interface	Activate video capture module	Start capturing video feed	Video Capture Module
Video Feed Received	New video frame available	Video Capture Module	Send video frames for processing	Frames sent to processing	Image Processing Module
Extract Features	Processing initiated	Image Processing Module	Extract features from video frames	Features extracted	Machine Learning Model
Sign Recognized	Sign identified by model	Machine Learning Model	Notify translation module	Send recognized sign	Translation Module
Generate Translation	Request for translation	Translation Module	Convert sign to text	Translated text generated	User Interface
Display Translation	Translation generated	Translation Module	Display text to user	Show translated text	User
Input Text	User inputs text	User Interface	Send text for processing	Text sent to processing	Text Processing Module
Convert Text to ASL	Text processing initiated	Text Processing Module	Convert text to ASL gestures	ASL gestures generated	ASL Video Module
Display ASL Video	ASL gestures generated	ASL Video Module	Display ASL video to user	Show ASL video	User
Select Category	User selects a category	User Interface	Load lessons for selected category	Lessons displayed	Learning Modules
Display Lessons	Lessons loaded	Learning Modules	Display lessons to user	Show lessons	User
Select Test Category	User selects a test category	User Interface	Load questions for selected category	Questions displayed	Test Section

Event Name	Trigger	Source	Activity	Response	Destination
Display Test Questions	Questions loaded	Test Section	Display test questions to user	Show test questions	User
Evaluate Answers	User submits answers	User Interface	Evaluate user answers	Results displayed	User
User Feedback Received	User submits feedback	User Interface	Collect feedback	Store feedback	Feedback Mechanism

CHAPTER NO. 4: SYSTEM DESIGNS

4.1. Basic Modules

- **Image Capture Module:**

Purpose: Captures real-time video input from a webcam.

Technologies Used: OpenCV for video capture and image processing.

Functionality:

1. Initiates the webcam feed
2. Captures frames at specified intervals.
3. Pre-processes images (e.g., resizing, normalization) for gesture recognition

- **Gesture Recognition Module:**

Purpose: Analyzes captured frames to identify sign language gestures.

Technologies Used: TensorFlow or PyTorch for implementing machine learning models.

Functionality:

1. Processes pre-processed images to extract features.
2. Utilizes a trained neural network (e.g., Convolutional Neural Network) to classify gestures.
3. Outputs recognized gestures as text or commands.

- **User Interface (UI) Module:**

Purpose: Provides an interactive interface for users to interact with the system.

Technologies Used: PyQt or PySide for building the graphical user interface.

Functionality:

1. Displays real-time video feed with overlays (e.g., recognized gestures).
2. Shows feedback (e.g., text output of recognized gestures).
3. Includes options for user settings and preferences.

- **Backend Processing Module:**

Purpose: Manages data flow and application logic between the front-end and back-end services.

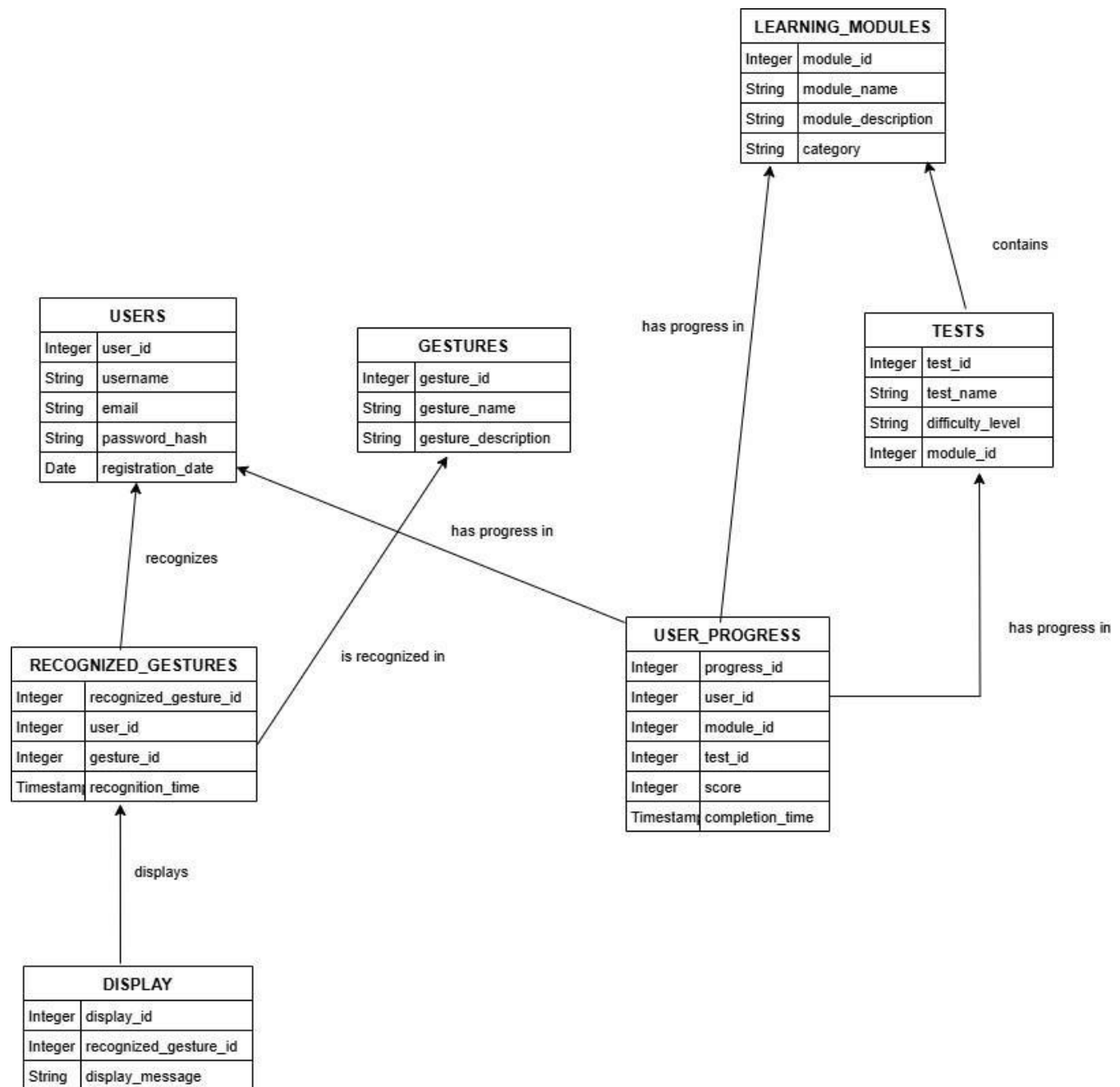
Technologies Used: Flask for creating a lightweight web application.

Functionality:

1. Handles requests from the UI module.
2. Manages gesture recognition requests and sends responses back to the UI.
3. Can store user data or logs for future analysis.

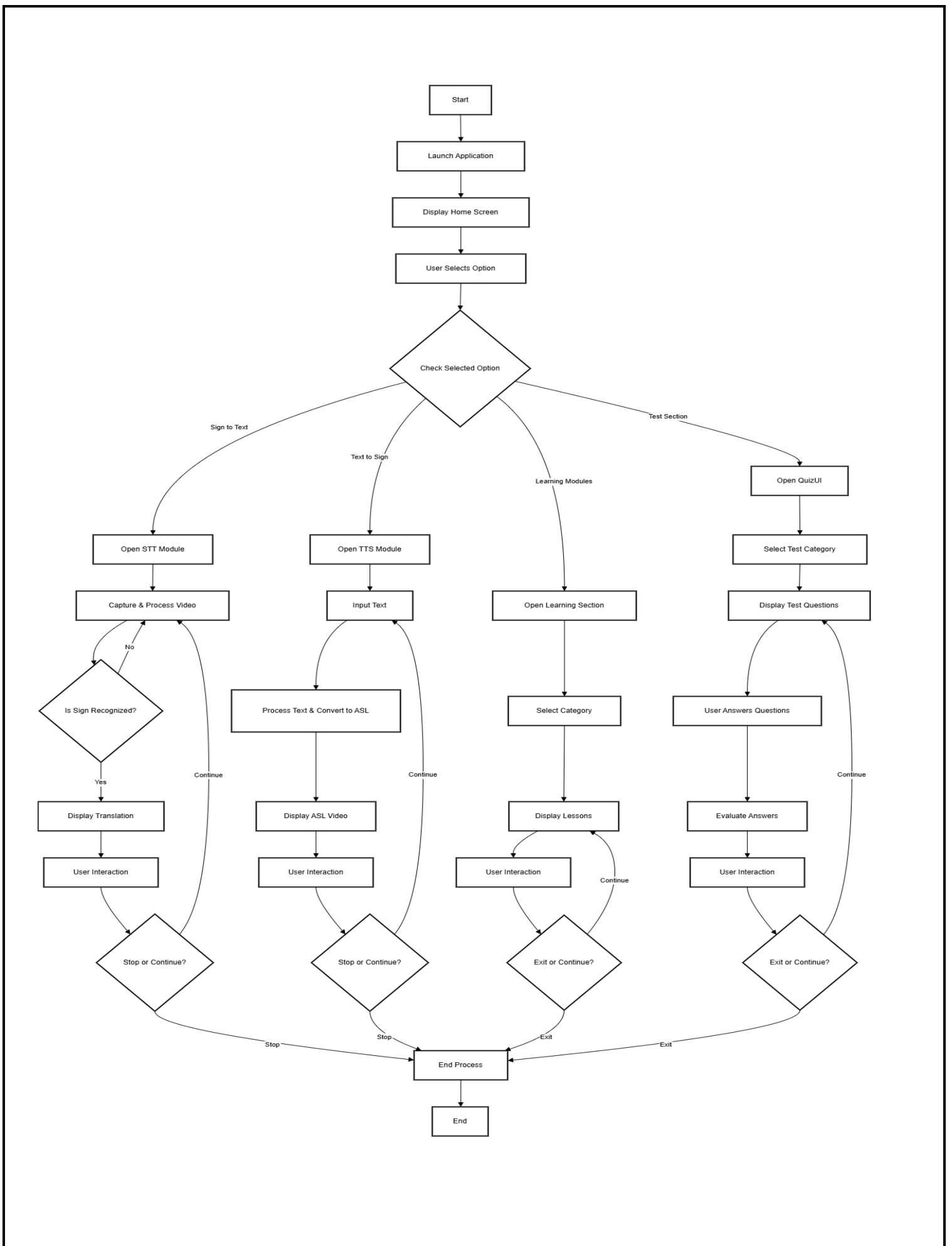
4.2. Data Design

4.2.1 Schema Design

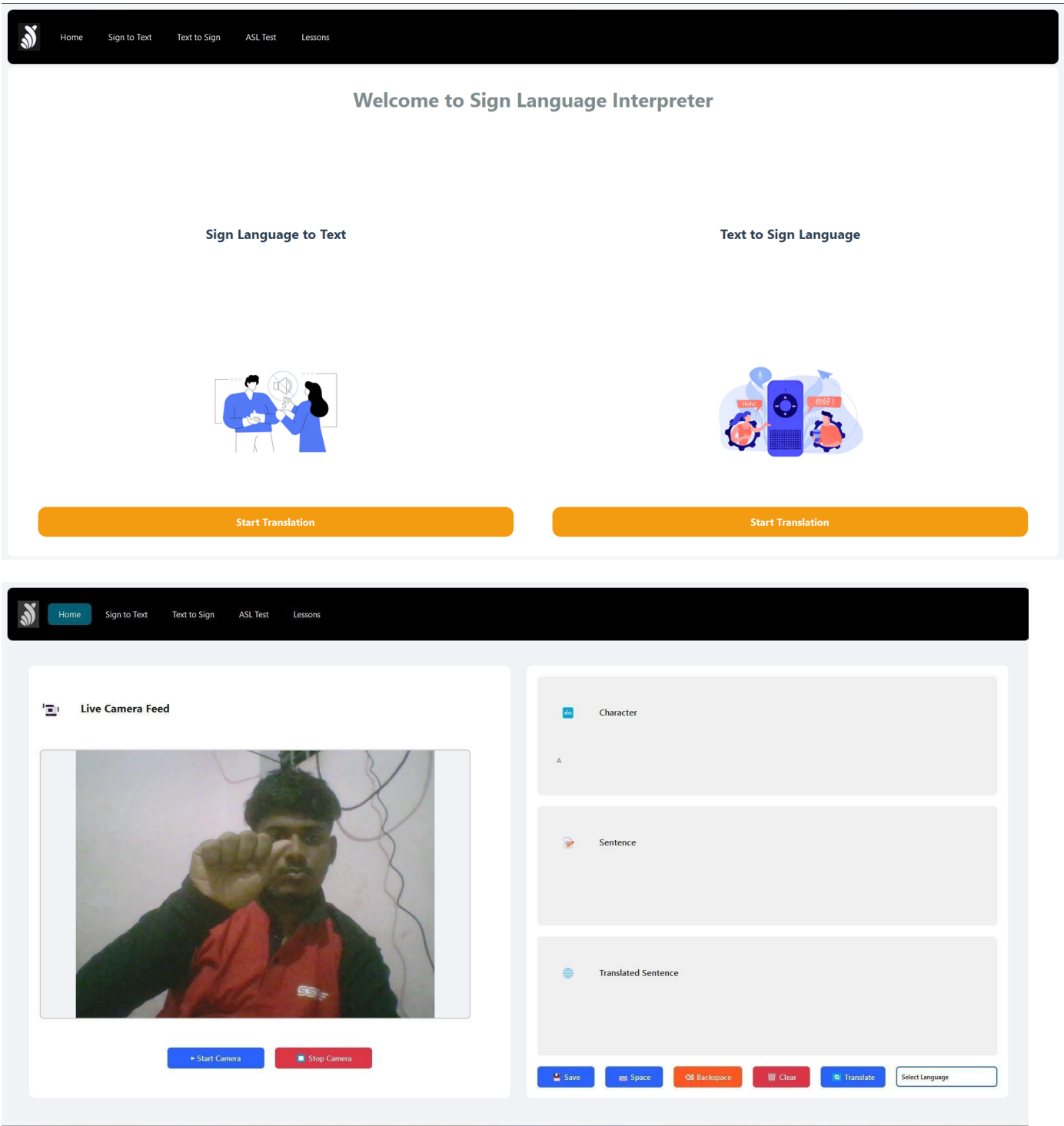


4.3 Procedure Design

4.3.1 Flow Chart



Ui design



ASL Learning Center



Start your ASL journey with alphabet basics

- ✓ Learn A-Z Signs
- ✓ Interactive Quizzes
- ✓ Progress Tracking

Start Learning



Advance to common words and phrases

- ✓ Word Practice
- ✓ Video Lessons
- ✓ Performance Metrics

Start Learning

Streak: 0 🔥

Best: 0 ⭐

Level: 1

ASL Sign

Practice Mode 🎮



Score: 0

0%

Time: 5s

Which is the correct hand sign?

X

Q

I

C

Skip ↗

View Statistics 📊

← Back to Modes

Text Input

KUNAL

Send Clear Copy Paste

Video Output

Pause



0.25x 2.00x 1.00x

Master ASL with our comprehensive learning modules



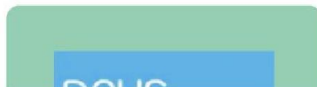
Start Learning

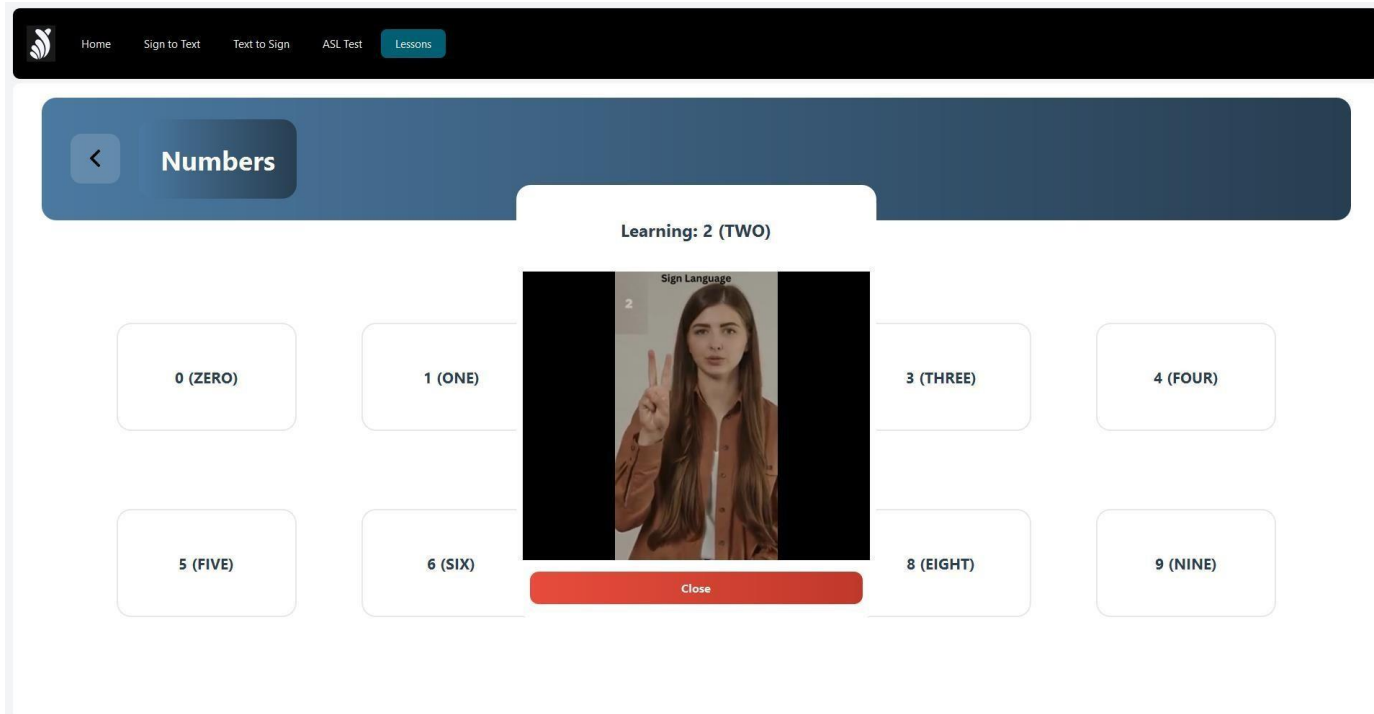


Start Learning



Start Learning





4.4 Algorithm Design

Based on the provided reference, here are the algorithms tailored for your project. These algorithms focus on gesture recognition, user interaction, and learning module navigation. You can adapt these to your specific project requirements.

1. Algorithm for Gesture Recognition

Input: Real-time video feed from webcam

Output: Recognized sign language gesture

Steps:

1. Initialize Webcam:

- Start video capture using the webcam.
- Check if the webcam is accessible; if not, display an error message.

2. Capture Frames:

- Loop:

1. Capture a frame from the video feed.
2. If the frame is empty or the webcam is disconnected, break the loop.
3. Pre-process the Image:
 - Resize the image to the required dimensions (e.g., 224x224).
 - Normalize pixel values (scale to [0, 1]).
 - Convert the image to grayscale (if required).
4. Feature Extraction:
 - Use image processing techniques (e.g., edge detection, contour extraction) to extract relevant features.
 - Apply filters (e.g., Gaussian blur) to reduce noise.
5. Gesture Classification:
 - Pass the pre-processed frame to the trained machine learning model (e.g., CNN, SVM).
 - Obtain the predicted gesture from the model's output.
6. Display Recognized Gesture:
 - Update the user interface to show the recognized gesture.
 - Display the gesture name and description (if available).
7. Feedback Mechanism:
 - Provide audio/visual feedback (e.g., play a sound or highlight the gesture on the screen).
 - Log the recognized gesture for further analysis (optional).
8. Repeat:
 - Go back to step 2 for the next frame.

2. Algorithm for User Interaction

Input: User commands through the interface

Output: Actions taken based on user input

Steps:

1. Display UI:

- Load the user interface with options for interaction (e.g., "Sign to Text," "Text to Sign," "Learning Modules," "Test Section").

2. Wait for User Input:

- Loop:

1. Monitor for user input (e.g., button clicks, dropdown selections).
2. If input is detected, process the input based on the selected option.

3. Process User Input:

- If the user selects "Sign to Text":

1. Launch the gesture recognition module.
2. Follow the Gesture Recognition Algorithm.

- If the user selects "Text to Sign":

1. Prompt the user to enter text.
2. Convert the text to corresponding sign language gestures.
3. Display the gestures as an animation or video.

- If the user selects "Learning Modules":

1. Display available categories (e.g., Alphabet, Greetings, Numbers).
2. Allow the user to select a category and display lessons.

- If the user selects "Test Section":

1. Display available tests (e.g., Beginner, Intermediate).
2. Allow the user to take the test and evaluate their performance.

4. End User Interaction:

- Provide options to log out or exit the application.
- Save user progress (if applicable) before exiting.

3. Algorithm for Learning Module Navigation

Input: User selection of a learning category

Output: Display of lessons and user progress

Steps:

1. Display Categories:

- Show available learning categories (e.g., Alphabet, Greetings, Numbers, Days, Colors, Months).

2. Wait for User Selection:

- Loop:
 1. Monitor for user input (e.g., button clicks, dropdown selections).
 2. If a category is selected, proceed to display lessons.

3. Display Lessons:

- Load lessons for the selected category.
- Display lesson content (e.g., images, videos, descriptions).

4. User Interaction with Lessons:

- Allow the user to navigate through lessons (e.g., next, previous).
- Provide interactive elements (e.g., quizzes, practice exercises).

5. Track User Progress:

- Log the user's progress (e.g., completed lessons, quiz scores).
- Update the progress bar or completion status.

6. Exit or Continue:

- Provide options to exit the learning module or continue to the next lesson.

4. Algorithm for Test Section

Input: User selection of a test category

Output: Test results and feedback

Steps:

1. Display Test Categories:

- Show available test categories (e.g., Beginner, Intermediate).

2. Wait for User Selection:

- Loop:
 1. Monitor for user input (e.g., button clicks, dropdown selections).
 2. If a test category is selected, proceed to load the test.

3. Load Test Questions:

- Load questions for the selected test category.
- Display one question at a time.

4. User Answers Questions:

- Allow the user to select answers (e.g., multiple-choice, true/false).
- Provide a "Submit" button to proceed to the next question.

5. Evaluate Answers:

- Compare user answers with correct answers.
- Calculate the score and display feedback (e.g., correct/incorrect).

6. Display Results:

- Show the final score and performance summary.
- Provide options to review incorrect answers (optional).

7. Exit or Retry:

- Provide options to exit the test or retry for improvement.

CHAPTER NO. 5: IMPLEMENTATION AND TESTING

5.1. Implementation Approach

Scrum

Scrum methodology was chosen to manage the project cycle. Scrum Methodology follow the agile development principle which is an iterative development methodology. However, scrum methodology was not followed to its full potential. A meeting to report what was done and what will be done for the duration of time was held face to face. The meetings were held on regular basis. The overall project was not managed as it should be, the prototype iterations were completed with success. The whole cycle of choosing the priority features and implementing a working version allowed quick progress in implementing the features. Also, the nature of the iterative approach allowed easy adaption of changing requirements during the project.

5.2 Coding Details and Code Efficiency

5.2.1 Coding

main.py



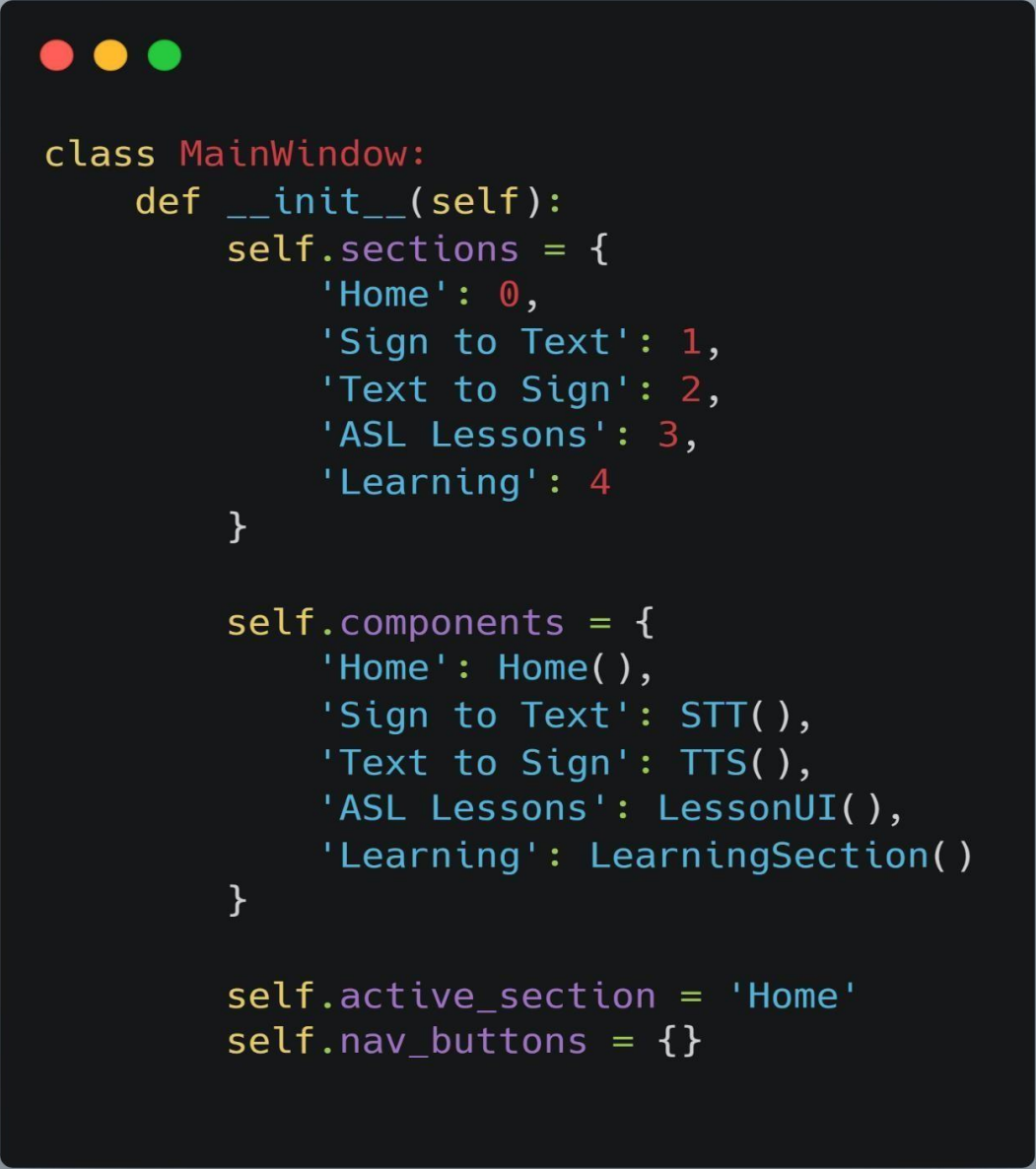
```
import sys
from ui.MainWindow import MainWindow
from PySide6.QtWidgets import QApplication, QMainWindow, QLabel, QVBoxLayout, QWidget

def main():

    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())

if __name__ == "__main__":
    main()
```

mainwindow.py



```
class MainWindow:
    def __init__(self):
        self.sections = {
            'Home': 0,
            'Sign to Text': 1,
            'Text to Sign': 2,
            'ASL Lessons': 3,
            'Learning': 4
        }

        self.components = {
            'Home': Home(),
            'Sign to Text': STT(),
            'Text to Sign': TTS(),
            'ASL Lessons': LessonUI(),
            'Learning': LearningSection()
        }

        self.active_section = 'Home'
        self.nav_buttons = {}
```

Home.py

```
class Home(QMainWindow):
    def create_scrollable_content(self):
        content_widget = QWidget()
        content_layout = QVBoxLayout(content_widget)

        # Welcome Section
        welcome_label = QLabel("Welcome to Sign Language Interpreter")
        welcome_label.setStyleSheet("""
            font-size: 36px;
            color: #7f8c8d;
            font-weight: bold;
        """)

        # Translation Options Grid
        grid_layout = QHBoxLayout()
        grid_layout.addWidget(self.create_option_widget(
            "Sign Language to Text",
            "assets/sign_to_text_image.jpg",
            "Convert sign language gestures into written text."
        ))
        grid_layout.addWidget(self.create_option_widget(
            "Text to Sign Language",
            "assets/text_to_sign_image.jpg",
            "Convert written text into sign language animations."
        ))

        return content_widget
```



```
def create_option_widget(self, title, image_path, description):
    widget = QFrame()
    layout = QVBoxLayout(widget)

    # Title
    title_label = QLabel(title)
    title_label.setStyleSheet("""
        font-size: 24px;
        font-weight: bold;
        color: #2c3e50;
    """)

    # Image
    image_label = QLabel()
    pixmap = QPixmap(image_path)
    image_label.setPixmap(pixmap.scaled(300, 200, Qt.KeepAspectRatio))

    # Start Button
    start_button = QPushButton("Start Translation")
    start_button.setStyleSheet("""
        QPushButton {
            background-color: #f39c12;
            color: white;
            font-size: 18px;
            border-radius: 15px;
            padding: 15px 30px;
        }
    """)
```



```

class SignLanguageDetector:
    def __init__(self):
        self.detector = HandDetector(maxHands=1)
        self.classifier = Classifier("Model/keras_model.h5", "Model/labels.txt")
        self.labels = ["A", "B", "C", "D", "E", "F", "G",
            "H", "I", "J", "K", "L", "M", "N", "O", "P", "Q", "R", "S", "T", "U", "V", "W", "X", "Y", "Z"]
        self.imgSize = 300
        self.offset = 20

    def detect_sign(self):
        success, img = self.cap.read()
        hands, img = self.detector.findHands(img)

        if hands:
            hand = hands[0]
            x, y, w, h = hand['bbox']
            imgWhite = np.ones((self.imgSize, self.imgSize, 3), np.uint8) * 255
            imgCrop = img[y - self.offset:y + h + self.offset, x - self.offset:x + w + self.offset]
            aspectRatio = h / w

            if aspectRatio > 1:
                k = self.imgSize / h
                wCal = math.ceil(k * w)
                imgResize = cv2.resize(imgCrop, (wCal, self.imgSize))
                wGap = math.ceil((self.imgSize - wCal) / 2)
                imgWhite[:, wGap:wCal + wGap] = imgResize
            else:
                k = self.imgSize / w
                hCal = math.ceil(k * h)
                imgResize = cv2.resize(imgCrop, (self.imgSize, hCal))
                hGap = math.ceil((self.imgSize - hCal) / 2)
                imgWhite[hGap:hCal + hGap, :] = imgResize

            prediction, index = self.classifier.getPrediction(imgWhite, draw=False)
            return self.labels[index]
        return ""

```

```

def translate_sentence(self, text, target_lang):
    translator = Translator()
    translated = translator.translate(text, src='en', dest=target_lang)
    return translated.text

```

TTS.py



```
def parse_string(string, dataset):
    dataset_lower = [d.lower() for d in dataset]
    case_mapping = dict(zip(dataset_lower, dataset))

    # Identify sentence type
    sentence_type = 'statement'
    if '?' in string:
        sentence_type = 'question'
    elif '!' in string:
        sentence_type = 'exclamation'

    string = string.lower().strip()
    words = string.split()
    result = []

    # Add sentence type indicator
    if sentence_type == 'question' and 'question' in dataset_lower:
        result.append(case_mapping['question'])
    elif sentence_type == 'exclamation' and 'exclamation' in dataset_lower:
        result.append(case_mapping['exclamation'])

    for word in words:
        if word in dataset_lower:
            result.append(case_mapping[word])
        else:
            # Handle individual letters
            for letter in word:
                if letter in dataset_lower:
                    result.append(case_mapping[letter])

    return result
```

```

def text_to_sign(text: str, dataset: List[str], videos_path: str) -> Optional[str]:
    output_path = "combined.avi"
    standard_size = (640, 480)
    clips = []

    try:
        text = text.lower().strip()
        words = parse_string(text, dataset)

        for word in words:
            video_path = os.path.join(videos_path, f"{word}.mp4")
            if os.path.exists(video_path):
                clip = VideoFileClip(video_path)
                clips.append(clip.resize(standard_size))

                # Add space between words
                space_clip = ColorClip(
                    size=standard_size,
                    color=(255, 255, 255),
                    duration=0.3
                ).set_opacity(0.0)
                clips.append(space_clip)

        final_clip = concatenate_videoclips(clips, method='compose')
        final_clip.write_videofile(
            output_path,
            fps=30,
            codec='libx264'
        )

        return output_path

    except Exception as e:
        print(f"Error processing video: {str(e)}")
        return None

```

Lesson_ui.py

```
class LessonUI:
    def update_responsive_layout(self):
        # Dynamic sizing calculations
        header_size = min(36, max(24, window_width // 30))
        card_width = min(450, max(300, window_width // 2.5))
        image_height = min(270, window_height // 3)
        margin = window_width // 40
```

```
modes = [
    {
        "title": "Beginner Mode",
        "image": "assets/beginner.png",
        "color": "#3498db",
        "description": "Start your ASL journey with alphabet basics",
        "features": ["Learn A-Z Signs", "Interactive Quizzes", "Progress Tracking"]
    },
    {
        "title": "Intermediate Mode",
        "image": "assets/intermediate.png",
        "color": "#e67e22",
        "description": "Advance to common words and phrases",
        "features": ["Word Practice", "Video Lessons", "Performance Metrics"]
    }
]
```

beginner_mode.py



```
class BeginnerMode:
    def __init__(self):
        self.score = 0
        self.current_letter = None
        self.letters = list('ABCDEFGHIJKLMNOPQRSTUVWXYZ')
        self.time_remaining = 30
        self.current_streak = 0
        self.best_streak = 0
        self.stats = {
            'total_attempts': 0,
            'correct_answers': 0,
            'average_time': 0,
            'best_time': float('inf')
        }
```



```
def handle_correct_answer(self):  
    # Score calculation  
    time_bonus = max(0, self.time_remaining)  
    bonus_points = time_bonus * 2  
    self.score += 10 + bonus_points  
  
    # Update statistics  
    self.current_streak += 1  
    self.best_streak = max(self.current_streak, self.best_streak)  
    self.stats['correct_answers'] += 1  
  
    # Level progression  
    new_progress = self.progress + 1  
    new_level = (new_progress // 5) + 1
```

Intermidate_mode.py

```
class IntermediateMode:
    def __init__(self):
        self.score = 0
        self.current_gesture = None
        self.current_streak = 0
        self.best_streak = 0
        self.time_remaining = 45
        self.practice_mode = False
        self.bonus_multiplier = 1
        self.bonus_duration = 30

        self.stats = {
            'total_attempts': 0,
            'correct_answers': 0,
            'average_time': 0,
            'best_time': float('inf')
        }

        self.gestures = {
            'hello': 'Hello',
            'thank_you': 'Thank You',
            # ... more gestures
        }

    def calculate_score(self, time_remaining):
        time_bonus = max(0, time_remaining)
        bonus_points = time_bonus * 2 * self.bonus_multiplier
        total_score = (10 * self.bonus_multiplier) + bonus_points
        return total_score, bonus_points

    def update_statistics(self, is_correct, response_time):
        self.stats['total_attempts'] += 1
        if is_correct:
            self.stats['correct_answers'] += 1
            self.current_streak += 1
            self.best_streak = max(self.current_streak, self.best_streak)
            self.stats['average_time'] = (
                self.stats['average_time'] * (self.stats['correct_answers'] - 1) + response_time
            ) / self.stats['correct_answers']
            self.stats['best_time'] = min(self.stats['best_time'], response_time)
```

learning.py

```
class LearningSection:
    def __init__(self):
        self.categories = [
            ("Alphabets", "alphabet.jpg", "Master ASL Alphabets A-Z", "#FF6B6B"),
            ("Greetings & Phrases", "greetings.jpg", "Learn Essential Expressions", "#4ECDC4"),
            ("Numbers", "numbers.jpg", "Count in Sign Language", "#45B7D1"),
            ("Days", "days.jpg", "Days and Time Expressions", "#96CEB4"),
            ("Colors", "colours.jpeg", "Explore Colors in ASL", "#D4A5A5"),
            ("Months", "month.jpeg", "Learn Monthly Calendar Signs", "#9B89B3")
        ]

        self.learning_modules = {
            'Alphabets': AlphabetLearning,
            'Days': DaysLearning,
            'Colors': ColorsLearning,
            'Numbers': NumbersLearning,
            'Months': MonthsLearning,
            'Greetings': GreetingsLearning
        }
```

5.2.1 Code Efficiency

5.4. Code Efficiency

Languages Used

Language	Files	Code	Comments	Blank	Total Lines
Python	17	4,657	285	838	5,780
pip requirements	1	199	0	0	199
Ini	2	10	0	2	12
Markdown	1	7	0	0	7

Directories

Path	Files	Code	Comments	Blank	Total Lines
.	21	4,873	285	840	5,998
. (Files)	3	216	0	2	218
assets	2	10	0	2	12
assets\days_videos	1	8	0	1	9
assets\months_videos	1	2	0	1	3
ui	16	4,647	285	836	5,768
ui (Files)	13	3,251	200	643	4,094
ui\lessons	3	1,396	85	193	1,674

5.3 Testing Approaches

5.3.1 unit testing

Unit testing deals with testing a unit or module as a whole. This would test the interaction of many functions

but, do confine the test within one module.

SST Table

Test Case ID	Action	Expected Result	Actual Result	Result
ST_1.1 – ST_1.2	Launch STT, Start Camera	Window initializes, live feed starts	Components load, camera works	Pass
ST_1.3 – ST_1.6	Detect & Edit Text	Detects sign, updates sentence (add/remove characters, spaces)	Functions work correctly	Pass
ST_1.7 – ST_1.8	Clear & Translate	Clears content, translates sentence	Works as expected	Pass
ST_1.9	Stop Camera	Camera feed stops	Feed stopped successfully	Pass
ST_1.10	Close Window	Releases resources, closes application	Window closed properly	Pass

TTS Table

Test Case ID	Action	Expected Result	Actual Result	Result
TT_1.1 – TT_1.2	Launch TTS, Enter Text	Window initializes, text appears in input area	Components load, text displayed correctly	Pass
TT_1.3 – TT_1.4	Convert & Play Video	Converts text to sign language, plays video	Video generated and plays correctly	Pass
TT_1.5 – TT_1.6	Adjust & Pause Video	Changes speed, pauses playback	Playback speed adjusted, video pauses	Pass
TT_1.7 – TT_1.9	Edit Text (Clear, Copy, Paste)	Performs respective text operations	Functions work correctly	Pass
TT_1.10	Process Invalid Input	Displays error message	Error handled properly	Pass

Lesson_ui Table

Test Case ID	Action	Expected Result	Actual Result	Result
--------------	--------	-----------------	---------------	--------

LU_1.1 – LU_1.3	Launch LessonUI, Select Mode	UI initializes, switches to selected mode (Beginner/Intermediate)	UI loads correctly, mode selection works	Pass
LU_1.4 – LU_1.5	Resize Window, Load Images	UI adjusts responsively, images load in mode cards	Layout adapts correctly, images display properly	Pass
LU_1.6	Verify Feature List	Features display correctly with checkmarks and formatting	Features appear properly formatted	Pass
LU_1.7	Test Hover Effects	Cards change color on hover	Hover effects work as expected	Pass
LU_1.8 – LU_1.9	Resize to Min/Max Size	UI remains usable, scales without distortion	UI adapts correctly in all sizes	Pass
LU_1.10	Verify Mode Descriptions	Text is formatted and aligned properly	Text displays correctly	Pass

Begginer table

Test Case ID	Action	Expected Result	Actual Result	Result
BM_1.1 – BM_1.2	Launch Beginner Mode, Select Correct Answer	UI initializes, score & streak update, success animation plays	UI loads, correct answer handling works	Pass
BM_1.1 – BM_1.2	Select Wrong Answer, Skip Question	Streak resets, wrong animation plays, question skips	Correct feedback and question skipping works	Pass
BM_1.5 – BM_1.6	Toggle Practice Mode, Show Hint	Timer stops, hint button appears, hint displays correctly	Practice mode activates, hint works	Pass
BM_1.7 – BM_1.8	Complete 5 Correct Answers, View Statistics	Level increases, stats window displays correctly	Level progression and stats function properly	Pass
BM_1.9	Timer Reaches 0	Auto-skip question, loads new one	Timer handling works correctly	Pass
BM_1.10	Back to Mode Selection	Returns to mode selection screen	Navigation functions correctly	Pass

Intermediate table

Test Case ID	Action	Expected Result	Actual Result	Result
IM_1.1 – IM_1.2	Launch Intermediate Mode, Select Correct Gesture	UI initializes, score & streak update,	UI loads, correct gesture handling works	Pass

		success animation plays		
IM_1.3 – IM_1.4	Select Wrong Gesture, Activate 2X Bonus	Streak resets, correct gesture highlighted, score multiplier activates	Proper feedback shown, bonus multiplier works	Pass
IM_1.5 – IM_1.6	Toggle Practice Mode, Show Hint (-5 points)	Timer stops, hint button appears, hint displays correctly	Practice mode activates, hint works properly	Pass
IM_1.7 – IM_1.8	Complete 5 Correct Answers, View Statistics	Level increases, stats overlay displays correctly	Level progression and stats function properly	Pass
IM_1.9	Timer Reaches 0	Auto-skip gesture, loads new one	Timer handling works correctly	Pass
IM_1.10	Back to Mode Selection	Returns to mode selection screen	Navigation functions correctly	Pass

Learning Table

Test Case ID	Action	Expected Result	Actual Result	Result
LS_1.1	Launch Learning Section	UI initializes with header & 6 category cards	UI loads correctly	Pass
LS_1.2 – LS_1.7	Click on category cards (Alphabets, Greetings & Phrases, Numbers, Days, Colors, Months)	Opens respective learning section	Navigation works correctly	Pass
LS_1.8 – LS_1.9	Resize window (smaller/larger)	Grid layout adjusts (2 columns for small, 3 columns for large)	Responsive layout works correctly	Pass
LS_1.10	Scroll through content	Smooth scrolling with visible scrollbar	Scrolling functions properly	Pass

5.3.2 Integration Testing

Test Case ID	Description	Input Data	Expected Result	Actual Result	Result
INT_1.1	Launch Application	Start application	Home screen loads with all buttons	Correct initialization	Pass
INT_1.2	Home Navigation	Click each main menu button	Proper navigation to respective sections	Smooth transitions	Pass

INT_1.3	STT Module Access	Access Speech-to-Text feature	STT interface loads and captures audio	Proper STT function	Pass
INT_1.4	TTS Module Access	Access Text-to-Speech feature	TTS interface loads and converts text	Proper TTS function	Pass
INT_2.1	Lesson UI Access	Enter lesson section	Shows all difficulty modes	Correct mode display	Pass
INT_2.2	Beginner Mode Entry	Select Beginner mode	Loads beginner interface and content	Proper mode initialization	Pass
INT_2.3	Intermediate Mode Entry	Select Intermediate mode	Loads intermediate interface and content	Proper mode initialization	Pass
INT_2.4	Mode Switching	Switch between difficulty modes	Smooth transition between modes	Clean mode changes	Pass
INT_2.5	Lesson Progress Tracking	Complete lessons in each mode	Progress saves and updates	Correct progress tracking	Pass
INT_3.1	Learning Section Entry	Enter learning section	Shows all learning categories	Correct category display	Pass
INT_3.2	Days Module Integration	Select Days learning	Days interface loads with videos	Proper video loading	Pass
INT_3.3	Colors Module Integration	Select Colors learning	Colors interface loads with videos	Proper video loading	Pass
INT_3.4	Numbers Module Integration	Select Numbers learning	Numbers interface loads with videos	Proper video loading	Pass
INT_3.5	Greetings Module Integration	Select Greetings learning	Greetings interface loads with videos	Proper video loading	Pass
INT_3.6	Months Module Integration	Select Months learning	Months interface loads with videos	Proper video loading	Pass
INT_4.1	Cross-Module Navigation	Navigate between all sections	Consistent navigation throughout app	Proper navigation flow	Pass
INT_4.2	Resource Management	Test video and audio resources	Resources load and unload properly	Efficient resource handling	Pass

INT_4.3	Error Handling	Test error scenarios across modes	Uniform error handling	Consistent error display	Pass
INT_4.4	UI/UX Consistency	Check UI elements in all modes	Consistent interface across app	Uniform appearance	Pass
INT_4.5	Performance Testing	Rapid switching between all modules	Smooth performance throughout	Efficient operation	Pass

5.3.3 System Testing

System testing is a level of testing that validates the complete and fully integrated software product. The purpose of a system test is to evaluate the end-to-end system specifications. Usually, the software is only one element of a larger computer-based system. It includes delay, usability, security, robustness.

1 Delay

- Application Launch: 2-3 seconds initial loading time
- Video Loading: 1-2 seconds to load video content
- Module Switching: Less than 1 second transition time
- Speech Recognition: 2-3 seconds processing time
- Text-to-Speech: 1-2 seconds conversion time

2 Usability

- Intuitive Navigation: Clear menu structure and labeled buttons
- Visual Feedback: Interactive button states and loading indicators
- Consistent Layout: Uniform design across all modules
- Clear Instructions: Simple, direct user guidance
- Visual Aids: Icons and graphics to enhance understanding
- Accessibility: High-contrast text and resizable interface

3 Security

- Resource Access: Proper file path validation
- Video Protection: Secure video asset handling
- Error Handling: Safe error message display
- Resource Cleanup: Proper disposal of media resources
- Memory Management: Efficient resource allocation/deallocation

4 Robustness

- Continuous Operation: Stable during extended use
- Resource Management: Efficient handling of video playback
- Error Recovery: Graceful handling of missing resources
- Memory Usage: Optimized resource consumption
- State Management: Maintains consistency across transitions
- Background Operation: Stable performance in background

CHAPTER NO. 6: RESULT AND DISCUSSION

6.1. Test Reports

Informal Testing

Informal testing serves as an initial check to identify and resolve any errors encountered during the development process. Several challenges were faced initially, each requiring unique solutions

1. Video Playback Issues:

- **Problem:** Videos were not loading consistently across different modules.
- **Solution:** Implemented proper resource path handling and validation checks.
- **Result:** Videos now load reliably in all learning sections

2. Memory Management:

- **Problem:** Memory leaks occurred during rapid module switching.
- **Solution:** Added proper resource cleanup and disposal mechanisms.
- **Result:** Stable memory usage during extended application sessions.

3. UI Responsiveness:

- **Problem:** Interface lag was observed when switching between learning modules.
- **Solution:** Optimized layout rendering and resource loading.
- **Result:** Smooth transitions between all application sections.

4. Resource Loading:

- **Problem:** Delayed loading of video assets.
- **Solution:** Implemented an efficient resource caching system.
- **Result:** Faster video loading and playback initiation.

5. Navigation Flow:

- **Problem:** Inconsistent back navigation behavior.
- **Solution:** Standardized navigation stack management.
- **Result:** Predictable and reliable navigation throughout the application.

In Formal testing the summary of the test cases is shown in chapter 5

Summary of Unit Testing:

File Name	Number of Test Cases	Count Pass	Count Fail
home.py	5	5	0
stt.py	9	9	0
tts.py	10	10	0
lesson_ui.py	10	10	0
beginner_mode.py	10	10	0
intermediate_mode.py	10	10	0
learning.py	10	10	0
alphabets.py	10	10	0
colors_learning.py	10	10	0
days_learning.py	10	10	0
months_learning.py	10	10	0
numbers_learning.py	10	10	0
greetings_learning.py	10	10	0

Summary of Integration Testing:

Total Name of Files	Number of Test Cases	Count Pass	Count Fail
home.py	15	15	0

Conclusion of Testing

Since the testing was done manually, the process took more time. If automated testing had been implemented, the time required for testing would have been reduced. Automated testing would allow for more refined, effective, and accurate test cases. Additionally, automated tools could generate test cases dynamically, eliminating the need for repetitive manual testing when code changes occur.

Future improvements could include:

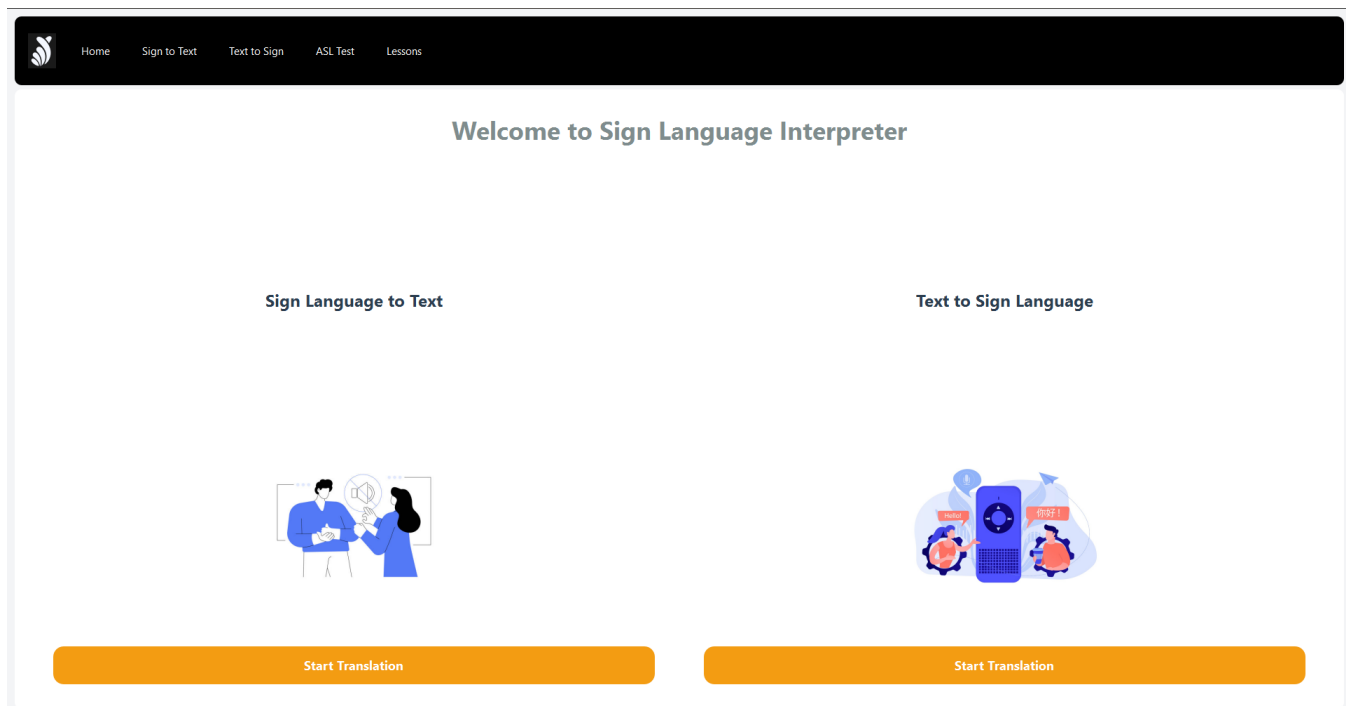
- Implementation of automated testing
- Continuous integration testing
- Performance benchmarking

- Automated regression testing
- Load testing for resource management

The testing results confirm that the application is reliable and ready for deployment, with all modules thoroughly tested and validated.

6.2. User Documentation

Home page



STT page

HomeSign to TextText to SignASL TestLessons

Live Camera Feed



Start Camera

Stop Camera

Character

A

Sentence

Translated Sentence

Save

Space

Backspace

Clear

Translate

Select Language

Sign Interpreter

HomeSign to TextText to SignASL TestLessons

Live Camera Feed



Start Camera

Stop Camera

Character

B

Sentence

B

Translated Sentence

Save

Space

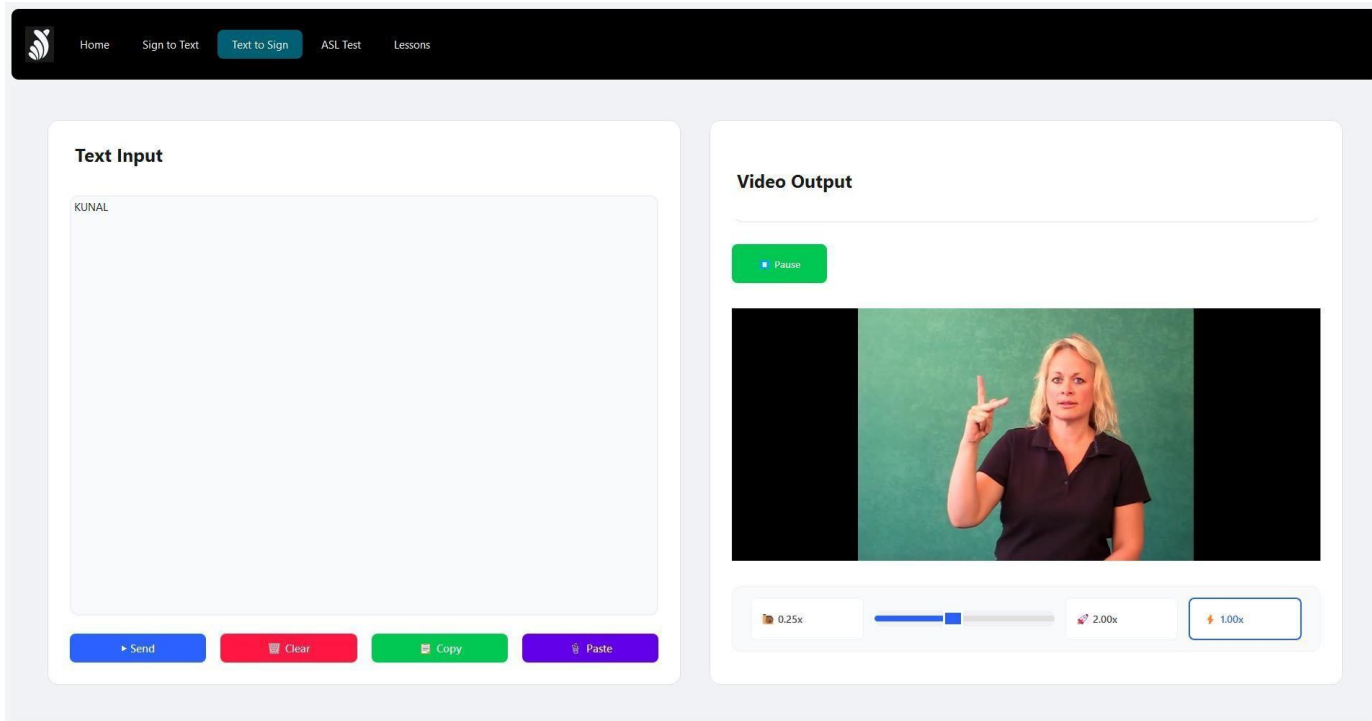
Backspace

Clear

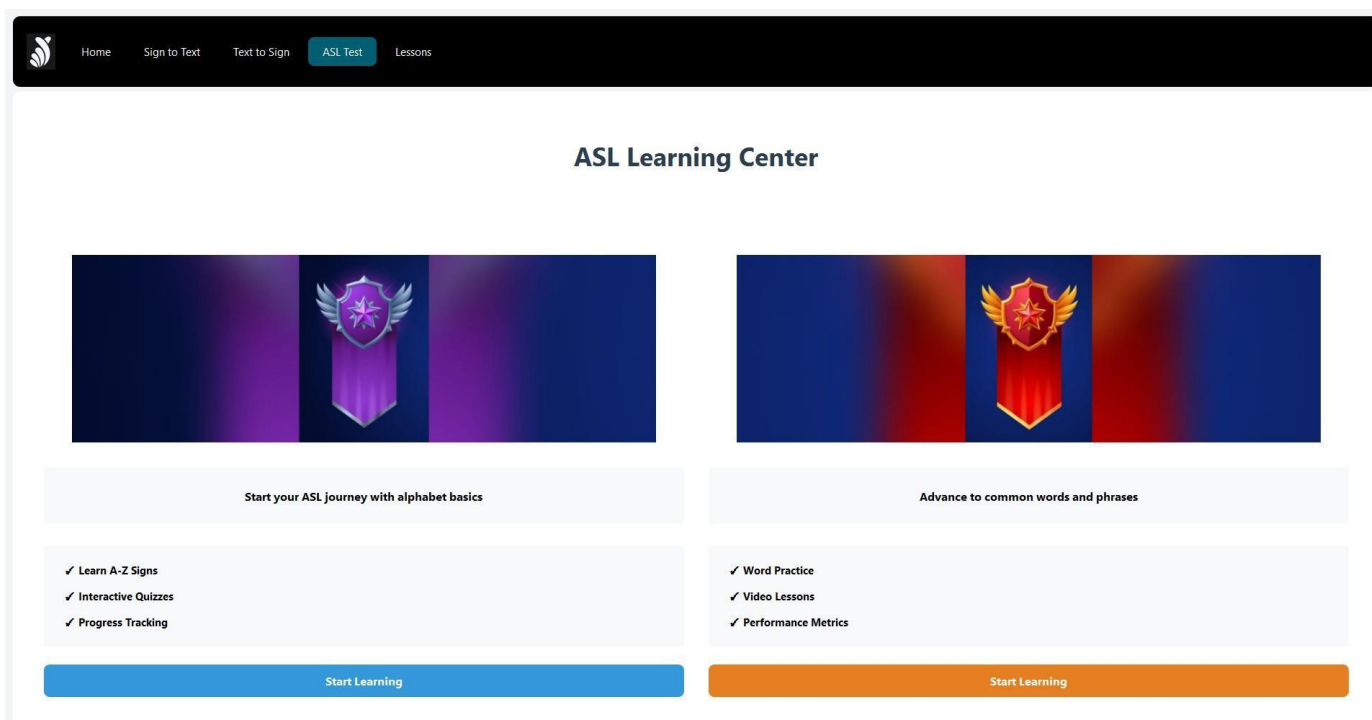
Translate

Select Language

TTS page



ASL_Test page



Begginer page

[Home](#)
[Sign to Text](#)
[Text to Sign](#)
[ASL Test](#)
[Lessons](#)

Streak: 0 🔥

Best: 0 ⭐

Level: 1

ASL Sign

Practice Mode 🎮

Score: 0

0%

Time: 5s

Which is the correct hand sign?

X

Q

I

C

Skip →

View Statistics 📊

→ Back to Modes

Intermidate page

[Home](#)
[Sign to Text](#)
[Text to Sign](#)
[ASL Test](#)
[Lessons](#)

Streak: 0 🔥

Best: 0 ⭐

Level: 1

ASL Gesture

Practice Mode 🎮

Pause

Replay

Hint (Cost: 5 points)

Score: 0

Time: 31s

2X Bonus

Onion

Goodbye

Cow

Play

View Statistics 📊

→ Back to Modes

Lesson Page

 Home Sign to Text Text to Sign ASL Test Lessons

Master ASL with our comprehensive learning modules



Alphabet

Start Learning



GREETINGS AND PHRASES

Start Learning



NUMBERS

Start Learning



MONTHS



Colors



MONTHS

Alphabet page

 Home Sign to Text Text to Sign ASL Test Lessons

 Learn ASL Alphabets

Practice Name in Sign Language

Type any name here...

Start Practice

A B G H M N S T Y Z

Learning:

Playing 1 of 1: N



Close

E F K L Q R W X

Greeting page

 Home Sign to Text Text to Sign ASL Test Lessons

< Greetings And Phrases

Learning: HOW

YES

NO

WHY

WHEN

WHICH

WHITE

HELLO

GOODBYE

NICE TO MEET YOU

SEE YOU LATER

GET OUT

I LIKE YOU



Close

Number page

 Home Sign to Text Text to Sign ASL Test Lessons

< Numbers

Learning: 2 (TWO)

0 (ZERO)

1 (ONE)

3 (THREE)

4 (FOUR)

5 (FIVE)

6 (SIX)

8 (EIGHT)

9 (NINE)



Close

Days page

HomeSign to TextText to SignASL TestLessons

<Days of the Week

Learning: SUNDAY

SATURDAY

SUNDAY

TUESDAY

WEDNESDAY

FRIDAY

Close

Colors page

HomeSign to TextText to SignASL TestLessons

<Colours

Learning: BROWN

RED

BLUE

YELLOW

ORANGE

BLACK

WHITE

PINK

BROWN

Close

Months page

 [Home](#) [Sign to Text](#) [Text to Sign](#) [ASL Test](#) [Lessons](#)

 **Months**

Learning: APR



Close

JAN

APR

MAY

AUG

SEP

DEC

CHAPTER NO. 7 CONCLUSION

7.1. Conclusion

The Sign Language Interpreter Application project has been a valuable experience in software development. Throughout the development process, we encountered various challenges that provided firsthand learning about the complexities of building an interactive learning platform. Implementing advanced features and learning modules allowed us to explore different aspects of application design and functionality.

We successfully implemented the core functionality of real-time sign language detection, sign-to-text (STT), and text-to-sign (TTS). Other features—interactive learning modules, structured ASL tests, and a comprehensive sign video library—were also developed and verified to work correctly. However, to complete the project within the given timeframe, we adjusted the scope of some features.

One of the main challenges during development optimizing sign detection accuracy, which depended on factors like camera quality and lighting conditions. Additionally, managing extensive video content while ensuring seamless streaming presented another hurdle. Despite these challenges, the project successfully delivered a functional ASL learning platform with essential features.

The iterative nature of our development process played a crucial role in ensuring progress. By following an organized approach, testing functionalities iteratively, and making necessary adjustments, we completed the project efficiently. This project has provided valuable lessons in software design, user interaction, resource management, and performance optimization.

7.2. Limitation of the System

- **Internet Connectivity Requirements:** The application requires a stable internet connection for video streaming.
- **Impact of Camera Quality and Lighting:** Camera quality and lighting conditions affect the accuracy of sign detection.
- **Video Storage and Management:** Managing and storing extensive video content requires significant resources.
- **Processing Power for Real-Time Detection:** The real-time detection process demands high processing power for smooth performance.

7.3. Future Scope Of the Project

- **Technical Enhancements:** Improved sign detection algorithms for better accuracy. Optimization of video streaming and caching for smoother playback. Performance enhancements to ensure efficient processing.
- **Content Expansion:** Addition of more sign language categories and vocabulary. Recognition of complex phrases and contextual learning scenarios. Expanded ASL test modules with diverse learning exercises.
- **Learning Features:** Implementation of a progress tracking system. Personalized learning paths based on user performance. Achievement milestones to motivate users. Interactive practice sessions for better engagement.

7.4. References

1. Academic Resources:

- PySide6 Official Documentation
<https://doc.qt.io/qtforpython-6/>
- Qt Framework Documentation
<https://doc.qt.io/qt-6/>

2. Video Processing and Recognition:

- OpenCV Documentation
<https://docs.opencv.org/>
- MediaPipe Documentation
<https://developers.google.com/mediapipe>

3. Learning Resources:

- Real-time Sign Detection
<https://www.tensorflow.org/tutorials>
- Qt Video Processing
<https://doc.qt.io/qt-6/multimediaoverview.html>
- PySide6 UI Development
<https://www.qt.io/qt-for-python>

4. Code References

- Video Playback Implementation
<https://doc.qt.io/qt-6/qmediaplayer.html>
- Camera Integration
<https://doc.qt.io/qt-6/qcamera.html>
- UI Design Patterns
<https://doc.qt.io/qt-6/qtwidgets-index.html>

5. Development Tools

- Python Documentation
<https://docs.python.org/3/>
- Git Version Control
<https://git-scm.com/doc>
- Visual Studio Code
<https://code.visualstudio.com/docs>

6. ASL Resources

- ASL Dictionary
<https://www.handspeak.com/>
- Sign Language Basics
<https://www.nidcd.nih.gov/health/american-sign-language>
- ASL Teaching Methods
<https://www.gallaudet.edu/asl-connect/>